38. 下列程序的运行结果为：
1. x=300
2. x=600
3. x=10
4. x=300

```
#include <iostream.h>
static int x=500;
void main()
{ auto int x=300; cout<<"1.x="<<x<<endl;
  ff(); ff(); cout<<"4.x="<<x<<endl;
}
void ff(){ x+=100; cout<<"2.x="<<x<<endl;}
void ff(){ int x=10; cout<<"3.x="<<x<<endl;}
```

main()以前定义x的分别是文件下面全局变量使用.
ff在main()函数中2定义6是x. ...

39. 在用new运算符创建数组时，不能为该数组指定初始值. 在创建对象数组时，初值将为默认.
int *p = new int(10); //OK. 表动分配一个整型的存储单元，初始值为10. int *p=new int;
int *p= new int[10]; 分配10个整型存储单元.
int *p= new int[5](10); //error.

40. C++规定构造函数不能是虚函数.

41. while(p!=NULL){...} ← 可能要再加个指针
while(p->next!=NULL){...} 则未指向最后个
do{ }while(); 若为空指针，可能有问题

42. 在定义数组时，系统将自动为其分配一块连续的内存空间，此时，数组名指向这块空间的起始点，且在程序中不可改变. 即一个数组名相当于一个地址常量.

43. 结构体成员可以嵌套，即结构体的成员可以为另一个结构体变量.

44. 每个指针变量都有一个相应的基类型，用以说明这个指针所指向的地址中存放的指向的数据类型.
int **pp; //pp为一个指针变量，所指向的空间中存放的是另一个指针. 这个指针指向一个整型变量.
pp为一个指向指针的指针.

int (*pf)(); pf为一个指向函数为返回值的指针，该函数返回值为int型数值，无形参表.
char (*pc)[7]; 指向一维数组的指针
int *p=10; //error. p为一个指针变量，不能赋常量值.
char *p="string"; //OK. 因为计算机先给字符串"string"分配一个空间，并将首地址赋给p. 但p指向的是单个字符
cout<<p<<'\t'<<*p<<endl; ⇒ string S

45. 条件运算符 d1?d2:d3
功能为：先计算d1的值，且进行判断，若d1非0，则表示d1的值为d2的值，否则为d3的值.

46. 逗号表达式 d1, d2, d3, d4. 从左至右依次计算，整个逗号表达式的值，类型也取后一表达式决定

47. 语句可以嵌套，即if体、else if体或else体内都可包含其他if语句. 在if语句嵌套的情况下，else总与最近的一个没有与else配对的if配对. 因此一个if总伴有一个else.

45. 函数调用时所采用的值传递、引用传递.
值传递时，调用函数所实参使用常量、变量值或表达式值，被调用函数的形参使用变量值. 调用时系统先计算实参表达式的值，再将实参的值按顺序对应地赋给形参. 即对形参进行初始化. 形参改变不影响实参.

46. 下述3种情况都将调用拷贝构造函数建立一个对象初始化为一个对象.
1) 明确地表示由一个对象初始化另一个对象时.
2) 当对象作为函数实参传递给函数形参时（值传递方式）
3) 当对象作为函数返回值时

47. 构造函数和析构函数没有返回值，且定义它们时不需要指定 void. 构造函数可重载，不能带缺析构函数不能重载，无参数.
程序中不能直接调用构造函数. 在创建对象时

系统原动调用和选定调

析和选择可被调用，也可由系统调用。

No.

Date

Chapter 6.

6.3 Class Point

```
{ double x, y;
public:
  Point () { x=y=0; }
  Point (double x1, double x2) { x=x1; y=x2; }
  double r() const { return sqrt(x*x+y*y); }
  double theta() const
  { return atan(y/x); }
  double distance (const Point &P) const
  { return sqrt((P.x-x)*(P.x-x)+(P.y-y)*(P.y-y)); }
  Point relative (const Point &P)
  { return Point(x-P.x, y-P.y); }
  bool is_above_left (const Point &P)
  { return (x<=P.x && y>P.y); }
};
```

关于虚函数、抽象类

```
#include <iostream.h>
class Figure
{public:
  virtual void draw() const =0;
  virtual void input_data()=0;
};

class Rectangle : public Figure
{ double left, right, bottom, top;
public:
  void draw() const
  { ... //画矩形
  }
  void input_data()
  { cout<<"请输入矩形左上角、右下角坐标";
    cin>>left>>top>>right>>bottom;
  }
};
```

```
class Line : public Figure
{ double x1, x2, y1, y2;
public:
  void draw() const { ... }
  void input_data() { ... }
};

class Figuregroup
{ Figure *figure_buf[100];
  int num;
public:
  Figuregroup() { num=0; }
  void display()
  { for (int i=0; i<num; i++)
      figure_buf[i]->draw();
  }
  void input_figures_data()
  { for (int j=num; j<100; j++, num++)
    { int shape;
      do { cout<<"请输入图形种类
      (0:线段, 1:矩形, -1:结束)";
      cin>>shape; } while (shape!=
      0 && shape!=1 && shape!=-1);
      if (shape==-1) break;
      switch (shape)
      { case 0: figure_buf[num]=new Line;
        break;
        case 1: figure_buf[num]=new
          Rectangle; break; }
      figure_buf[num]->input_data();
    }
  }
};
```

实现链表表示法逆序

No.

Date

```
struct Node { int content; Node *next; };
void reverse (Node *&head)
{ Node *p=head, *q; head=NULL;
  while (p)    // (p!=NULL)
  { head=NULL;
    q=p->next;
    p->next=head;
    head=p;
    p=q;
  }
}
```

之前创建该类对象.

2. 构造函数的调用.

创建时, 自动调用. 可根据调用 which one.

class A { --- public:

A(); A(int i); A(char *p); };

A a1; // 默认 ⇔ A a1=A();

A a2(1); // 调用 A(int i). ⇔ A a2=A(1);

A a2=1;

A a3("ABC"); // A(char*p);

// 相当于 A a3=A("ABC"); A a3="ABC";

A a[4]; // a[0]...a[3] 默认不构造出来.

A b[3]={A(), A(1), A("CDD")};

A *p=new A; // 1实.

A *q=new A(4);

A *p1=new A("ABCDE");

A *p2=new A[20]; 创建动态数组时, 只能调用无对象和默认不构造函数.

*程序中可通过类的构造函数来创建一些临时对象.

A a; a=A(10); // 创建了一个临时对象并赋值给a. 上面创建了一个临时对象并调用构造函数 A(int)对其初始化, 然后把该临时量赋值给 a. 临时对象在包含它的表达式计算完后就会自动消亡.

3. 成员初始化表.

常量成员, 引用成员.

class A

{ int x; const int y; int &z;

public:

A(): z(x), y(1) { x=0; }

};

或 A(): x(0), z(x), y(1) { } 效果一样.

成员的初始化次序由它们在类定义中说明次序决定.

*当类中有常数据成员或引用数据成员时, 若类中定义了构造函数, 则一定要用成员初始化表对其也行初始化. 若没有构造函数, 则编译程序会给该类生成一个默认构造函数. 来创建对象.

4. 析构函数.

~<类名>(); 无参数, 无返回值.

当对象消亡时, 在系统收回它所占的存储空间之前, 应该析构函数都会自动被调用. (常用于归还动态资源) 不能重载.

一般情况下不需定义析构函数. 但只有对象创建后申请了一些资源且未归还, 则在对象消亡时应归还.

在对象创建时, 系统会为对象分配一块存储空间来存储对象的非静成员. 但对于指针类型的非静成员, 系统只分配了存储该指针所需的空间. 并未为该指针所指向的空间. 对象自己管理申请. 在对象消亡时, 系统收回的, 是指针成员本身存储空间, 所指空间需对象自己归还. 在析构函数中归还(堆空间).

*有些情况, 我们并不撤消对象, 只是归还所申请的资源, 可显示的调用析构函数. S.~String();

6.4 2 成员对象的初始化

class A

{ int m;

public: A() {m=0;}

A(int m1) {m=m1;} };

class B

{ int n; A a;

public: B() {n=0;}

B(int i) {n=i;}

B(int i, int j): a(j) {n=i;} };

B b1, b2(1), b3(1,2);

成员对象的初始化由成员对象类的构造函数来实现.

默认情况下 (b1, b2(1)) 调用默认构造函数.

b1.a.m, b2.a.m 都初始化为0.

b3.a.m 初始化为2

P12. 调用对成员对象构造函数，析构函数的调用

6.4.2 拷贝构造函数

创建对象时，用同类的另一个对象对其初始化，即调用拷贝构造函数

```
class A
{ ... public: A(); // 默认构造函数
  A(const &A&S); // 拷贝构造函数
};
```

三种情况会调用 copy 构造函数

① 定义对象时：A a1; // 调用默认的构造函数
A a2(a1); // or A a2=A(a1); A a2=a1;

② 把对象作为形参传给函数.
void f(A x); A a; f(a);

③ 把对象作为返回值.
return a;

*若在类定义中未给出 copy 构造函数，则编译器将会隐式地为其提供一个 copy 构造函数，进行成员拷贝初始化.（对成员对象，调用成员对象类的 copy 构造函数）

*类定义中包含成员对象时，其拷贝初始化也成员对象类的拷贝构造函数完成. 系统提供的隐式 copy 构造函数会先调用成员对象的 copy 构造函数. 自定义的 copy 构造函数不会去调用成员对象的 copy 构造函数，显式地指出.

未指出 → 默认构造函数.

6.5 const 成员函数

成员函数 { 修改对象状态.
获取对象状态.

```
class A
{ ...
  void f() const { ... } };
```

f 即为一个 const 成员函数. 在其函数体中不能修改被操作成员的值！

加上 const：表示它是一个获取对象状态的成员函数.

但是 class A

```
{ int *p;
  public: ...
    void f() const { *p='A'; //OK
    strcpy(p,"ABCD"); } //OK };
```

(p的值才被改)

const 成员函数还对常生对象的成员之间调用的合法性进行判断.

```
class A
{ int x,y;
  public: void f() const { ... }
    void g() { ... } };
A b; const A a;
b.f(); b.g(); a.f(); //OK
a.g(); // error.
```

*常生对象，常用于函数的形参类型，防止在一个对象作为参数时，为指向它的引用或指针类型，而修改实参对象. void f1(const A *pa);
void f1(const A &a);
f中不能调用对象本身或 a 的非 const 成员函数，从而不会改变实参对象的状态.

6.6 静态成员

同一个类的对象共享静态数据.

```
class A
{ int x,y;
  static int share;
  public:
    A() {x=y=0;}
    void increase_all() {x++; y++; share++;}
    int sum_all() const { return x+y+share; }
  ...
};
```

访问结构成员：student A[10]; A[i].name;
↑ 成员访问操作符
左也为结构类型或类

strcpy(A[i].name, "张三");

对结构成员的访问受通过结构变量名来受限。不同结构类型的成员名应可相同。与非结构成员可相同。

结构作用域。

②整个赋值。相同的结构类型之间。

3.初始化。

student A={5, "张三"};

注：定义结构类型时，不能对成员初始化。不占内存。

4.存储 连续的存储空间， sizeof(student) 或 sizeof(A)

5.函数传递结构类型。

void display(student s){...}

display(A)

默认参数传递方式为值传递。 指针，引用

5.4 指针类型

描述内存地址，指针为内存地址的抽象表示。

1.指针类型的定义：

指向变量的指针类型 和所指变量的指针变量同时定义。

int *p; int *p, q; q为int类型。
所指向类型的类型，可以是基本的C++类型。
p为 int* 类型。

typedef int* pointer; pointer p, q;

2.取地址操作符 &

int *p, q; p=&q;

对于常相变量、函数，不用&，数组变名和函数名本身即表示它们在内存中的首地址。

空指针：常量0，或NULL(stdio.h 或 cstdio 中)

3.指针运算

①赋值 int *p1, *p2, x;
double *p3, y;
p1=&x; p2=p1; p3=&x //error
p2=p3; //error p1=null, p2=0

注①对指向指针变量，也应将首地址赋给它。

char *p; p="ABCD";

②任何类型的指针都可赋给void* 类型的指针变量。定义时可给出初始化。 int x; int *p=&x.

(2)访问指向的数据（间接访问操作符*，->）

注：对一个指向结构类型变量的指针变量，访问其成员

student a, *pa; pa=&a;

pa->name; (*pa).name; a.name;

(3)一个指针 +/- 一个整常数 （得到的是同类型指针）

常用此指针方式来访问数组元素。

int a[10], *p; p=a; //或 p=&a[0];

访问 a[i]时，可用 p[i]或 *(p+i)

(4)同类型指针相减，结果为整型值。

(5)两个同类型的指针比较（内存地址大小）

(6)指针的输出。 int x=1; int *p=&x;

cout<<p; // 输出x的内存地址，p的值。

cout<<*p; // 输出x的值

注：char str[]="ABCD";

char *p=str; // *p=&str[0];

cout<<p; // 输出的是字符串 ABCD

cout<<*p; // 输出A.

cout<<(void*)p; // 内存地址。

强制类型转换

5.4.2 指针作为形参类型

1.向函数传递实参的地址

C++默认的参数传递方式为值传递（数组除外），形参的值不为对应实参的值，为创建仅与实参的值，可以把形参定义成指针类型。在函数调用时，把实参的地址传给函数。在函数中通过实参的地址间接地改变实参的值

void SWAP(int *pa, int *pb)

{ int temp=*pa;

*pa=*pb; *pb=temp; }

向函数传递参数的地址，除了实现在函数中修改实参的值外，主要作用是提高参数传递的效率，节省内存空间，提高效率。

int max(int x[], int num) {…x[i]…} 等价于 int max(int *x, int num) {…*(x+i)…}

可用 *x[i]

2. 指向常量的指针

const int *p; int x; p=&x;

*p=1; // error. 不能改变p所指变量的值。

int f(const int *p); 则不能通过这个函数改变形参的值，目的是使传递效率提高。

const int x=0;

int *p; p=&x; // error. ← 指向变量的指针，不能指向常量

指针类型的常量：int *const p=&x;

必须有初始化，p的值不能改（内存地址），但*p可改

3. 指针作为返回值类型

int *max(const int x[], int num)

{ int max_index=0;

for(int i=0; i<num; i++)

if(x[i]>x[max_index]) index=i;

return (int*)&x[max_index]; }

5.4.3 指针与动态变量

有些数据元素个数在程序运行前无法确定，必须在程序运行时，根据具体情况来确定。

动态变量 ← 动态内存管理。 分配在堆中

自动变量 → 局部变量。 ← 编译程序已知其存在内存空间分配在栈中

动态变量无名字，需通过指针来访问

1. 创建：new <类型名>

在程序运行时，在堆区中开出一个类型为<类型名>指定的动态变量，结果为该动态变量的地址（指针）

int *p; p=new int; *p=1;

new int[m][n]… 动态数组。

结果为首地址，类型由编译决定

int *p; p=new int[20];

int (*q)[20]; q=new int[30][30];

⇔ typedef int A[30]; A *q;

2. 撤消：

C++运行期间，动态变量不会自动消亡，函数调用时创建的动态变量，返回后仍然存在且可使用。 内存泄漏

delete p; (p指向对象) delete[] q; (q指向对象)

3. 动态变量的应用——链表

解决由多个同类型的具有顺序关系的元素构成的复合数据

①动态数组

int n; int *p; cin>>n;

p=new int[n]; 一般而言数组不能这样做。

for(int i=0; i<n; i++) cin>>p[i];

sort(p, n);

②若输入时不能确定个数，可用链表来做。

链表用于表示由若干个（个数不定）同类型的元素所构成的具有线性结构的复合数据。

链表中元素除了本身的数据外，还包含一个（或多个）指针。

单链表、多链表， 内存不必连续

单链表由结点构成，数据+指针域， 头指针。

Struct Node 结构体成员可以是C++中任何数据类型，void除外，但可有指向本结构的指针

{ int content; Node *next; }; 指向的指针类型

Node *head=null; 指向的指针类型

一、在链表中插入一个新结点（值为a的结点）

1. 链表为空，即 head=null 的情况

Node *p=new Node; p->content=a;

head=p; p->next=null;

2. 链表非空，插在表头：

Node *p=new Node; p->content=a;

p->next=head;

head=p;

3. 插在表尾.

```
Node *p=new Node; p->content=a;
for(Node *q=head; q->next!=null; q=q->next);
q->next=p; p->next=null;
```

4. 插在第i(i>0)个结点的后面.

```
Node *p=new Node; p->content=a;
Node *q=head; int j=1;
while(j<... && q->next!=null)
{q=q->next; j++;}
if(j<i) cout<<"没有第i个结点"<<endl;
else {p->next=q->next; q->next=p;}
```

二. 在链表中删除一个结点. head!=null 情况.

1. 删除表中第一个结点.

```
Node *p=head; head=head->next;
delete p;
```

2. 删除表中最后个结点.

```
Node *p1=head, *p2=NULL;
while(p1->next!=NULL)
{p2=p1; p1=p1->next;}
(保证删除第2个表元)
if(p2->next!=null) {p2->next=null; delete p1;}
else {head=null; delete p1;} 可一次性.
```

3. 删除第i(i>0)个结点

```
Node *p1=head, *p2=NULL; int j=1;
while(j<i && p1->next!=NULL)
{p2=p1; p1=p1->next; j++}
if(i==i) {p2->next=p1->next; delete p1;}
else cout<<"没有第"<<i<<"个结点。"<<endl;
```

这样做不能用于删除第1个结点

```
if(i==1) {Node *p=head; head=head->next;
          delete p;}
else {int j=1;
      while(j<i-1 && p->next!=NULL)
```

No.

Date

```
if(p->next!=null)
{Node *q=p->next; p->next=q->next;
 delete q;}
else cout<<"没有第"<<i<<"个结点."<<endl;
```

三. 在链表中检索某个值a. (第i个?)

```
int index=0;
for(Node *p=head; p
index++;
if(p->content==a) break;}
if(p!=null) cout<<"第"<<index<<"个结点的
值为:"<<a<<endl;
else cout<<"没有找到值为"<<a<<的结点!"<<endl;
```

5.4.4 指针与数组

指针来访问数组元素可提高数组元素的效率

```
const int N=100; int a[N];
for(int *p=&a[0], *q=&a[N-1]; p<=q; p++)
{...}
```

1. 数组a的首地址.

(1) 一维数组. 首地址可由a[0]或&a[0]获得. a表示第0个元素的内存地址的不能修改. 如 q=... //error.

```
void f(int *a, int num) int a[],
{...*(a+i)...} 或 a[i].
```

调用时. int a[10]; f(a,10) 或 f(&a[0],10)

(2) 二维数组. (看2维)

int b[2][10]; ⇔ typedef int A[10]; A b[2];

内存首地址为b或&b[0]. 表示变量b[0]的地址, 类型为 A* int (*p)[10]

b[0], &b[0][0] 表示 b[0][0]的地址, 类型为 int*

这里虽然b(或&b[0])与b[0](或&b[0][0])表示同一个内存地址, 但指向不同的指针类型.

和二维数组作为一个参数时, 实际传递的是第一行的地址

void g(int (*p)[10], int lines)

{ ... *(*(p+i)+j) ... 或 p[i][j] ... }

调用时 int b[20][10]; g(b, 20) 或 g(&b[0], 20)

2. 主函数 main 的参数 P176.

5.4.5 函数指针

1. 指向函数的指针

指针变量可以指向函数. 格式: double (*fp)(int);

<所指函数的返回类型> (*<指针变量名>)(形参表);

double *fp(int); ← 声明为一个返回值为 double* 的函数 fp.

double f(int x); fp=&f; 或 fp=f;

typedef double (*FP)(int);

FP fp;

可通过函数指针调用其所指函数 fp(10); 或 (*fp)(10);

2. 用函数指针作函数参数

C++允许在调用一个函数时, 将一个函数作为参数传递给被调用者, 被调用函数的形参定义为一个函数指针类型, 实参为一个函数的地址.

double integrate(double (*fp)(double), double a, double b) {...}

调用时, integrate(sin, 0, 1);

5.5 引用类型

指针: 可访问对象变量, 指向函数参数的传递参数, 解决函数返回多个值的问题, 提高对数组元素访问的灵活性和效率. 可靠性小.

1. 引用类型的定义

int x=0;

int &y=x; // 引用没有自己专门的内存空间, 和变量 x 共享 x 的内存空间. 和变量同为 0 的别名

y=2; cout << x; cout << y; ...

当不当初始化, 访问引用实际上访问的是被引用的变量.

引用类型的变量定义之后, 不能再引用其他变量.

2. 引用类型作为函数的形参类型.

引用类型的形参与相应实参占用相同内存, 改变引用类型形参的值, 相应实参的值也会随着改变.

void swap(int &x, int &y);

int a=0; int b=4;

swap(a, b);

把形参定义为引用类型可提高参数传递的效率.

void f(const int &x) ... 所引用 x 的值不能被改变

P182. 引用与指针的区别:

① 可实现通过一个变量访问另一变量. 引用 → 直接访问形式, 指针 → *间接访问形式.

② 引用类型不能再引用其他类型. 安全.

③ 实参. (引用 ← 变量名). (指针 ← 变量地址)

3. 引用作为函数返回值类型

int &max(int x[], int n);

int main()

{ int a[10]; cout << max(a,10); 这与对没有引用类型的也可操作.

max(a,10)+=1; // 可修改 a[10] 中的最大元.

若一个函数的返回值类型为引用类型, 则该函数不应该返回对其局部变量的引用.

No.

Date

第五章 构造类数据类型.

5.1 枚举类型.

1. 定义：由用户自定义的简单类型，值集由程序定义。
   罗列举出其值集中的每个值。
   定义格式 enum Day {sun, Mon, Tue};
   枚举值对应整型数. 0, 1, ···
   也可自定义 对应的 enum Day {sun=7, mon=1, ···}
   使用：变量，形参，函数返回值等。
   Day some_day; enum {V1, V2} a, b;

2. 枚举类型运算
   ① 赋值. Day d1, d2; d1=sun; d2=d1;
   Day d3; d3=V1; // error.
   枚举值可赋值给整型变量，反之不可.
   Day d1, d2; d1=10; //error.
   d2=(Day)3; // 强制类型转换.
   ② 比较. 系统首先会将它转化为整型数. Mon>Tue 结果false.
   ③ 算术运算，转化为整型，结果为算术类型.
   Day d; d=d+1; //error. d+1的结果为整型数，不能赋值. d=(Day)d+1;

注：不能对枚举类型的值直接进行I/O. 间接方式.
Day d; cin>>d; //error. cout<<d; //error
提高易读性，保证正确性.
for(Day i=sun; i<=Sat; i=(Day)(i+1))
{···} d=10; //error.

5.2 数组类型.

一种由固定多个同类型的元素按一定次序所构成的数据类型，用户自定义. 一维. 多维.

5.2.1 一维数组.

线性次序关系的数据所构成的复合数据. ← 线性表.

1. 定义: int a[10]; 元素个数固定.
   一维数组类型定义与其变量的定义同时给出.
   元素类型除void外都可.
   typedef int A[10]; A a;

2. 操作的访问数组元素 ① 作为参数传给函数
   a[0] 访问第0个元素.
   下标越界. C++不报出，结果不可以!

3. 一维数组要显示初始化.
   int a[10]={1, 2, 3}; 不足补0. (自动)
   int b[]={2, 5, 7}; 可省长度. 3个.

4. 一维数组的存储.
   编译程序会在内存中为其分配连续的存储空间.
   sizeof(a) 求计算数组所占内存大小.

5. 同类数传递一维数组.
   int max(int x[], int n) {···} 数组元素个数
   调用时，只需名，长度. max(a, n);
   注：数组作为函数参数传递时，实际传递的是数组的首地址，函数的形参不会分配内存空间，它共享实参数组的内存空间. 一旦数组值所发生的将导致实参数的变化.

6. 字符数组. (字符串)
   C++中，通常用char的一维数组来表示字符串类型.
   char a[10]; 最多可存9个字符.
   结束标记 '\0'.
   定义字符数组时，长度大1，赋值用'\0'
   * 把一个字符数组作为函数时，调用者只需提供数组名即可. 不需个数.
   * 操作：① 访问元素 ② 作为参数传递
   ③ 输入和输出 cin>>a; cout<<a;
   输入操作会在字符数组的最后自动放上'\0'.
   char s[20], i;
   cin>>s;
   for(i=0; s[i]!='\0'; i++)
   {if(s[i]>='a'&&s[i]<='z') s[i]=s[i]-'a'+'A';}
   cout<<s<<endl;

*字符数组要注意初始化：

char S[10] = {'h', 'e', 'l', 'l', 'o', '\0'};

char S[10] = {"hello"};

char S[10] = "hello";

char S[] = "hello"; ← 这样定义长度为6.

* 标准库中提供对字符串进行处理的函数，

在 cstring 或 string.h 中.

(1) 计算长度 strlen(a);

所计算的字符个数不包括'\0'.

(2) 字符串复制.

注：字符串之间不可直接赋值 如 char a[10], b[10];

a[10] = "hello"; //error b[10] = a[10]; //error.

strcpy(a, b) 表示把b复制到a所指向的内存中 自动加'\0'.

strncpy strncpy(a, b, n) 表示把把b的n个字符复制到a中. b的字符个数小于n. 在后面加上'\0'.

否则只复制到n. 后不加'\0'.

返回值为a的内存地址. 一般不使用.

(3) 字符串拼接

Strcat(a, b); Strncat(a, b, n);

把b的字符串拼接到a后面. 返回a的地址.

(4) 字符串比较.

strcmp(a, b); Strncmp(a, b, n);

返回值为int型. 0表示相等. 负数表示b大. 正. a大.

字典顺序比较大小.

5.2.2 二维数组.

1. 定义: int A[10][5];

元素类型可以任意，除void. (10行5列.

typedef int A[10][5]; A a;

可将二维数组看成以一维数组为元素的一维数组.

typedef int B[5]; B A[10];

2. 操作: ①访问元素 ②作为实参传给函数.

A[9][4] 表示第10行，第5个元素.

A[i] 访问第i+1行.

3. 初始化.

int a[2][3] = {{1, 2, 3}, {4, 5, 6}};

int a[2][3] = {1, 2, 3, 4, 5, 6};

int a[2][3] = {{1, 2}, {3, 4}};

int a[2][3] = {1, 2, 3, 4}; ← 行用默

int a[][3] = {{1, 2, 3}, {4, 5}, {7}}; 3行.

4. 二维数组的存储.

连续的存储空间. 按行 sizeof(a)

5. 向函数传递二维数组. (不确定)

int sum(int x[][5], int lin)

{ int sum = 0;

for(int i = 0; i < lin; i++)

for(int j = 0; j < 5; j++)

sum += x[i][j];

return sum; }

调用时. cout << sum(a, 10) 确定可 或a[10][5];

列数必须写明.

C++中把二维数组传给一个函数时. 传的是内存首地址.

根据 x的内存首地址 + 行×列数 + 个 计算x[i][j]地址

5. 3 结构.

结构类型可表示固定多个类型的不同元素构成的复合数据. 用户自定义.

1. 定义: Struct Student

{ int numble; ← 结构成员.

char name[20]; };

结构成员类型不能为void及本类型! 但可以本类型的指针

可指定某些程序读件的类型. (读写. 形参. 返回值类型)

Student A[10];

结构类型名可省. 变量同时定义.

2. 操作:

①访问结构成员 同赋值 同作为参数传给函数.

同作为返回值类型返回给调用者.

第六章 数据抽象——类

6.1 从面向过程到面向对象.

2. 面向对象程序设计的定义:

面向对象程序设计是由若干对象构造程序.

*对象: 构成面向对象程序的基本计算单位. 由一些表指出这些数据所能实施的操作构成. 及接口.

*通信: 对象间的消息传递. 引起面向对象程序进行计算的唯一方式.

*类: 描述一组具有相同特征的对象.

*继承: 对象的一部分特征描述可从其他的类获得.

*抽象: 一个程序实体的描述强调而忽略程序实体的外部可观察到的行为, 与该程序实体的内部实现无关.

(1)过程抽象: 一些功能抽象为程序. 接口. 对类操作和操作的描述实现.

(2)类抽象: 此类指出中应. 把数据及施对类数据进行的操作作为一个整体. 数据描述中必含有施对其实施的操作.

*封装: 把一个程序实体的具体实现细节作为一个黑匣子对程序体的使用者隐藏起来的一种程序设计机制. 信息隐藏.

(1)过程封装: 子程序. 把操作封装.

(2)类封装: 类. 操作封装.

*模块化:

一个模块包含接口和实现. 内聚性高大, 耦合度最小.

*软件复用

面向对象. 类库和继承机制实现代码复用.

6.1.3 面向对象程序设计的基本内容.

1. 对象与类.

对象的特征(类指. 操作)人世对象的类描述.

对象包含的类指: 类指成员. 成员属性. 实例属性. 对象的局部属性.

对象的操作. 成员函数. 方法或消息处理过程.

类是对象的集合. 对象则是类的实例.

2. 继承: 一般 特殊 No 父类(基类) 子类(派生类)

3. 多态性. Date

(1)一名多用. 重载. 虚函数多重载. 操作符重载.

6.2 类的定义:

class <类名>

{ <成员描述> }; 类的成员: 数据成员. 成员函数

6.2.1 数据成员.

可以是常量成员和变量成员.

①类定义中指定的类指成员指示类的对象. 在创建对象前, 不占内存. 不能赋初值. (某些特殊数据成员除外)

初始化 -> 构造函数

②classA; // 声明.

classB { A a; // Error. 未A的定义.

A *p; // OK.

A &aa; //ok.

B b; //error. 未定义.

B *q; //OK B &g; //OK };

6.2.2 成员函数.

(1)在类定义中定义成员函数:

在类中定义. 当其以编译指令内联函数处理它们.

(2)类外定义. 在类中给出声明.

class Date { ··· };

void Date::set(int x, int y, int z) {···};

类成员函数可重载. (除析构函数外)

类定义放在头文件(.h)中. 类外定义的成员函数在实现文件(.cpp)中

6.2.3 成员的访问控制: 信息隐藏

public. private. (本类, 友元) protected (本类, 友元, 派生类)

指定为public访问控制的成员不仅成为类与外界的一种接口. 操作一个对象时, 只能通过访问对象类中的public成员来实现.

类作用域：类体中所标识符所属的类定义。

类对象使用类定义中所标识符时，通过对象名跟踪或类名跟踪。

class A { ... public: void f() {...} ... }; ... }; f为全局标识符时，同样。

6.3 对象

类在先于程序(运行前的程序中)。

动态所有由对象程序中对象构成。程序的执行通过对象间消息发送(请求实现)

6.3.1 对象的创建、析构

1. 直接方式：class A {... };

A a; A b[100];

全局对象、局部对象、成员对象

2. 间接方式（动态对象）

运行时，用new来创建，堆区

(1) 单个动态对象 A *p; p=new A;

delete p;

(2) 动态对象数组的创建和撤消

A *p; p=new A[100]; delete[]p;

注：用new创建的动态对象数组只能用默认构造函数进行初始化。delete中[]不能省。否则只调第一个对象的析构函数被调用。

6.3.2 对象的操作

class A {...}; A a;

A *p=new A; p->f(); a.f();

①函数对象类的成员函数

②取地址 ③赋值 ④作为实参给形参传递

⑤作为函数返回值

Date f(Date d);

Date yesterday, today, someday, *p;

someday=yesterday;

today=f(someday);

p=&yesterday;

赋值操作（一般情况）把右边对象的数据成员的值分别赋给左边对象的数据成员。

6.3.3 this指针

类定义中的数据成员（除静态）对该类型（对象都）有一个拷贝。A a,b; a,b分别拥有内存。在对数据成员函数中有一个指针。

每个成员函数都有个隐藏的指针类型的形参 this

A *const this; // 指针类型常量

class A

{ int a, b, c;

public: void f();

void g(int x) { a=x; f(); }; };

A x, y;

void g(A *const this, int x)

{ this->a=x; this->f(); }

调用时 x.g(1) ⟶ A::g(&x, 1);

一般this可省，但成员函数中要把this所指向的对象作为整体来操作（如取地址）应显示使用this）

6.4 对象初始化和清理前处理

初始化：数据成员 为对象申请资源

6.4.1 构造函数、析构函数

1. 构造函数：在对象类中定义或声明的与类同名、无返回值类型的成员函数。在创建对象时，构造函数会被自动调用。对象创建后不可再调用构造函数。可重载。不能定义的构造函数 → 默认构造函数

默认构造函数

①创建对象时，没有指定调用何种构造函数初始化，则→默认。

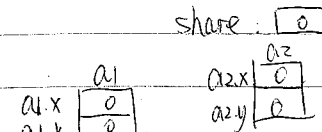
②类中未提供构造函数，编译程序在需要时隐式提供默认构造函数

③对象类中提供了构造一，则程序不默认 → 构造函数一般public，若为private，只能在本类及友

int A::share=0; // 静态类成员必须在类外部给出其定义，定义时可进行初始化

该类的所有对象共存在一个copy.

A a1, a2;



当通过一个对象引用多个静态类成员的值时，其他同类对象中看到的这个值都

2. 静态成员函数

静态成员函数只能访问静态成员.

```
class A
{ int x,y; static int share;
  public: A() {x=y=0;}
    static void set_share(int i) {share=i;}
    static int get_share() {return share;}
};
int A::share=0;
```

3. 静态成员的使用

除了在类的定义中可访问静态成员，类外都也可访问public静态成员

(1) 通过对象访问 A a; a.set_share(10);

(2) 通过类名限定访问 A::get_share();

类的静态成员与该类对象存在与否无关，静态的成员函数中没有隐藏的形参this.

6. 友元

在C++类定义中，可指定某个全局函数，某个其他类或某个其他类的成员函数来直接访问该类的private和protected成员. 友元函数、友元类、友元成员函数统称为友元

```
...
class A
{...
  friend void func();  // 友元函数
  friend class B;      // 友元类
  friend void C::f();
};
```

注：友元不具有传递性. B为A友元，C为B友元，若不显示指出，C非A友元.

第七章 操作符重载

对基本类型数据类型和相应的类型，C++预定义的操作符有固定含义. 对于类，除了如赋值、成员选择等外，C++没有定义它的含义. 由程序员定义操作符重载

规则：(1) 遵循已有操作符的语法. 重载操作符不改变操作符的优先级和结合性.

(2) 不能改变操作符的个数

(3) 尽量遵循有已有操作符的语义. (不是必需的)

(4) 重载操作符时，操作数至少有个为类，指针、引用及引用类型(除void)

(5) 以下5个操作符不能重载：".", ".*", "::", "?:", "sizeof".

不以作为非静态的成员函数重载操作符.

隐藏的参数出现，双目 ← 一个参数，自由 ← 不用.

1. 双目操作符重载：

一个参数，任意类型(除void)

1° 类中定义：

```
class A
{...
  <返回值类型> operator #(<类型><参数>)
  {... }
};
```

2° 类外定义(需类中声明)

```
class A
{...
  <返回值类型> operator #(<类型>);
};...
<返回值类型> A::operator #(<类型><参数>)
{...}
```

3° 使用 A a,b; a#b 或 a.operator#(b)

2. 单目操作符重载（不带参数）

前置

1° 类内定义 class B
{ ...
<返回值类型> operator #(){...}
};

2° 类外定义 class B
{ ...
<返回值类型> operator #();
};
<返回值类型> B::operator #()
{...}

3° 使用格式： B b; #b 或 b.operator#();

4° 为了使后缀的前置及后置++和--的重载，可定义一个带int型参数的操作符++、--，后置同此

```
class Counter
{ int value;
public:
  Counter() { value=0; }
  Counter& operator ++()   //前置++重载函数
  { value++; return *this; }
  const Counter operator ++(int)  //后置++重载函数
  { Counter temp=*this;
    ++(*this);
    return temp; }
};

Counter a,b,c;
b=++a; //调用不带参数的操作符++重载函数
c=a++; //带参数的操作符++重载函数
```

7.3 作为会员（友元）函数重载操作符

要求参数类型至少有一个为类、结构、枚举或其引用类型

1° 双目操作符

<返回值类型> operator #(<类型1><参数1>, <类型2><参数2>) {...}

若用到类中私有的数据类型，则必须以适当方式在类定义中给出声明

使用 a#b 或 operator#(a,b)

2 单目操作符：一般形式

Date

<返回值类型> operator #(<类型><参数>) {...}

<类型a>; #a operator #(a)

后缀时，在参数后加上 int.

※ 操作符 =, (), [], -> 只能作为成员函数来重载

其中可重载的操作符既可作为成员函数也可作为全局函数来重载

有选择地，操作符必须以全局函数来重载，才能满足要求

7.4 几个特殊操作符的重载

7.4.1 赋值操作符 =

C++编译程序会为每个新类定义一个隐式的赋值操作符重载函数，这个对成员函数进行赋值操作，着重于数据成员，通常为赋值成员对象，调用该成员对象的赋值操作符。

※ 赋值操作隐含两个过程：先通过析构当前对象的状态，后以对象的状态

※ 对于包含成员对象的类来说，用户定义的赋值操作符重载函数不会自动去调用成员对象的赋值操作符重载函数，必须要在自定义的~显示指出

```
class A {...};
class B
{ A a; int x,y;
public: ...
  B& operator=(const B& b)
  { a=b.a; x=b.x; y=b.y;
    return *this; }
};
B b; A a;  b=a;
b.operator=(a);
```

※ 一般地，需要自定义拷贝构造函数的类通常也需自定义赋值操作符重载函数

反复!!!

No.

①创建对象时，用另一个已存在的同类对象对其初始化，调用获取构造函数.

②两个已存在的对象，赋值，则调用赋值操作符=重载

A a; A b=a; // copy. A b(a);

b=a; // 赋值

7.4.2. 数组元素下标访问操作符

某些类的对象，表现成员成线性关系

class string

{ char *p;

public:

---

char& operator[](int i)

{ if (i>=strlen(p))

{ cerr<<"下标越界!\n"; exit(-1); }

return p[i]; }

};

操作: String s("abcdE");

for(int i=0; i<s.length(); i++)

if (s[i]>='a' && s[i]<='z')

s[i]=s[i]-'a'+'A';

对于数组元素的访问，运行系统不对下标越界进行检查，交给programmer. 类可进行下标越界检查

第七章 输入/输出 (I/O)

*在C++中，输入/输出不是语言所定义的成分，而是由操作系统所实现(编译器作为标准库函数来提供.

*C++中，输入/输出是一种基于字节流的操作. 进行输入时，输入的数据一个字节地从外设流入到计算机内存(内存). 输出，数据一个字节从内存流出到外设.

C++还提供了基于C++基本数据类型的I/O.

还可对输入/输出类的一些操作进行重载，使对自定义的对象也可进行I/O.

基于控制台的I/O { 标准输入设备输入(键盘)
标准输出设备(显示器)
标准错误输出设备(显示器)

基于文件的I/O { 从外存文件获得数据.
程序结果保存在外存文件中.

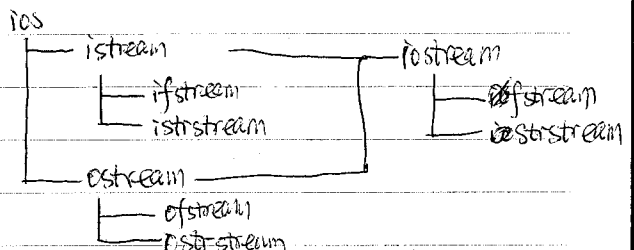
基于字符串的I/O { 字符串变量中获得数据
程序结果保存到字符串变量中

C++支持用户对象及过程的程序设计范型，输入/输出操作也可以这两种方式进行

*过程式I/O，通过函数库的I/O流来实现，可实现对基本数据类型的I/O操作.

*面向对象I/O，通过C++的I/O类库来实现

I/O类库提供的I/O操作在一些I/O类实现



istream 专用于输入操作的类

ostream 用于输出操作的类

iostream 既能输入又能输出

这些类在头文件<iostream.h>中定义的

* istream 重载了操作">>" 可用于基本类型数据的输入. "抽取".

输入时，首先创建一个istream类 或其派生类对象，然后调用该对象的提取操作">>"进行输入.

istream c; double x; int y=0;

c>>x; c>>y; c>>x>>y;

*ostream 重载了"插入"之"操作. 输出时，发生在

ostream c; int x=y=2; double z=5.4;

c<<x+y+z; c<<x; c<<x<<z;

* 为对基本数据类型进行输入/输出时，还可对">>"、"<<"进行重载，使得通过I/O类的对象可对用户自定义

对某指针加对象进行I/O操作.

10.2 控制台I/O.

10.2.2 基于I/O类库的控制台I/O.

1. 预定义的控制台对象.

cin (∈istream) cout, cerr, clog (∈ostream) 为输入:

利用预定义的I/O类库的类对象,可直接进行控制台的I/O,而不需要自己创建对象.

cin 对应标准输入设备

cout 对应用于输出程序正常运行结果的标准输出设备

cerr, clog ------ 输出错误信息的设备,

cerr 不对输出信息缓存,能快速响应.

*在 iostream.h 中声明. 该文件包含命令.

2. 输出.

任何基本数据类型的数据,指针,字符串(字符串指针)

*格式控制 (利用操纵符).

#include <iomanip>

endl 输出换行符,并执行 flush 操作.

flush 使输出缓存中的内容立即输出.

dec (默认) 十进制输出

oct 八进制

hex 十六进制

setprecision(int n) 设置要输出有效数字的位数或小数点后数字的位数.

setiosflags(long flags) / resetiosflags(long flags)

设置/取消输出格式, flags 的取值为 ios::scientific 以指数形式显示浮点数, ios::fixed (以小数形式)

#include <iostream>

#include <iomanip>

---

int x=10;

cout << hex << x << endl;

可通过 ostream 根据后面字符的操作进行输出.

ostream& ostream::put(char ch);

输出一个字符.

ostream& ostream::write(const char *p, int count)

输出指针p所指向的内存空间中 Date count个字节.

*输出操作往往带缓存,输出的内容不一定能立即在输出设备上出现. 用 flush 可实现立即输出.

cout << i << j << flush;

输入时,各个数据之间用空白符分开.

可通过操纵符控制输入行为.

setw 用于指定输入字符的最大个数.

cin >> setw(10) >> str; //把输入的前9个字符和一个'\0'放入str中.

*基于字节的对成员函数的输入.

输入一个字符 istream::get(char &ch);

istream::getline(char *p, int count, char delim='\n');

//输入一个字符串直到输入了count-1个字符或遇到delim指定的字符为止,并自动加上一个'\0'字符.

istream::read(char *p, int count);

//读入count个字节至p所指向的空间.

4. 输入/输出操作符 ">>" 和 "<<" 重载.

自定义类的对象的输入/输出操作. 重载操作符 ">>", "<<" 必须在类外定义,分为友元函数重载(友元形式)

class A

{ int x, y;

public:

friend ostream& operator << (ostream& out, const A& a);

};

ostream& operator << (ostream& out, const A& a)

{ out << a.x << ',' << a.y; return out; }

返回值引用类型. 则 cout << a << b << endl; 可行!

(cout为预定义的,因此不需要重新创建一个对象)

A a, b; ostream o;

o << a << endl << b;

*对于A的派生类B.

class B: public A

{ double z;

};

若未对类B重载"<<", 则 B b; cout<<b<<endl;

按类A对象进行: 已显示b.x, b.y.

```
class B: public A
{ double z;
public:
friend ostream& operator<<(ostream&, const B&);
};
ostream& operator<<(ostream &o, const B &b)
{ o<<(A&)b<<','<<b.z; return o;}
```

可解决直接输出B类对象问题. 但间接输出B时, --

10.3 文件I/O

用于永久性保存数据的设存储为外部介质的器. 如磁盘, 磁带, 光盘等.

保存数据的方式: 文件. 数据库.

* C++中, 把文件看成是由一系列字节所相成的字节流称为流式文件. 对文件中数据的操作(I/C)是个字节顺序进行.

* 对文件数据读写前, 必打开文件.

目的: 在程序内部和一个表示文件的实生/对象与外部的一个具体文件之间建立联系.

每个打开的文件都有一个内部的位置指针, 指明文件的当前读写位置.

* 读写时, 每读/写一个字节, 指针自动后移一个字节的位置.

* 读/写完毕后, 关闭文件. 把缓存在内存缓冲区的内容写到文件中, 并回收打开文件时申请的内存资源.

* 两种存储方式

文本(text): 可显示字符或以个按制字符'\r', '\n', '\t'. "的结构. 对于整数12345. 文本方式是7个ASCII码写入文件. 17个字节.

二进制方式(binary): 直接写入二进制字节, 一般用于存储非显式结构的数据.

把整数12345的内存的补码写入文件.

优点: 占用空间, 读写速度快. 不需转换.

缺点: 不便读. 不利于跨平台不同计算机平台上使用

10.3.2 基于I/C类库的文件I/O

在利用I/C类库中的类进行外部文件的输入/输出时, 程序用 #include <iostream>

#include <fstream>

1. 输出. (输出到文件中(写入))

首先创建一个ofstream类的对象. 然后与某个外部文件建立联系

* 直接法: ofstream out_file(<文件名>, <打开方式>);

* 间接: ofstream out_file;

out_file.open(<文件名>, <打开方式>);

文件名: 外部文件名(+地址)

打开方式: ios::out 打开一个外部文件用于操作. 若外部文件已存在. 先清除内容. 否则. 创建一个. 位置指针在文件开头

ios::app, 打开个外部文件用于添加(从文件末尾). 若文件不存在, 先创建才. 位置指针在文件末尾.

若打开方式忘写上面两种. 则为文本方式.

二进制打开 ios::out | ios::binary

ios::app | ios::binary

打开文件操作可能失败. 应判断一下!

```
if(!out_file) // 或 if(out_file.fail())
{---}          if(!out_file.is_open())
ofstream out_file("d:\\myfile.dat", ios::out);
if(!out_file) exit(-1);
int x; double y; ---
outfile<<x<<y<<endl;
outfile.put('A');
outfile.write("ABCDEFG", 7);
```

} ofstream后成员函数

* 文件输出操作结束时. out_file.close(); 关闭文件

```
write((char*)&x, sizeof(x));
A a;  write((char*)&a, sizeof(A));
write((char*)数组名, 数组长度*sizeof(元素类型));
```

2. 输入:

No.

Date

*首先创建 ifstream 类与外部文件建立联系.

(* 直接 ifstream in_file(<文件名>,<打开方式>);

* 间接 ifstream in_file;

in_file.open(<文件名>,<打开方式>);

打开方式 { ios::in : 打开一个外部文件用于读操作. 外部文件必须存在,否则打开失败. 文本方式(默认)

ios::in | ios::binary. 二进制方式打开

*对输入操作进行检查,判断文件是否已结束(数据被读完). 可调用 ios 类的成员函数 eof.

int ios::eof(); // 未结束. 非0 结束.

in_file.close();

3.输入/输出.

即打开一个既能读入也能输出的数据文件. 首创建一个 fstream 类的对象. 打开方式: ios::in | ios::out 可在文件任意位置读. ios::in | ios::app. 只能在文件末尾写.

fstream io_file("d:\\test.text", ios::in|ios::out);

4.随机存取文件.

显示指出读写位置.

输出文件(写文件).

ostream & ostream::seekp(<位置>); // 绝对位置

ostream & ostream::seekp(<偏移量>,<参照位置>);

streampos ostream::tellp(); // 获得指针位置

输入文件.

istream & istream::seekg(<位置>);

istream & istream::seekg(<偏移量>,<参照位置>);

streampos istream::tellg();

参照位置: ios::beg (文件头)

ios::cur (当前位置), ios::end (文件尾)

Chapter 5 Homework (1)

No.

Date

4. 从键盘输入某个星期内每天的最高和最低温度，然后计算平均最低和最高温度并输出

```
#include <iostream.h>
enum Day {Sun, Mon, Tue, Wed, Thu, Fri, Sat};
int main()
{ int max=0, min=0;
  int M[7], m[7];
  cout << "请输入一个星期内每天的最高气温:\n";
  for(Day i=Sun; i<=Sat; i=(Day)(i+1))
  {cin >> M[i]; max += M[i];}
  cout << "本星期平均最高温度为" << max/7;
  cout << "请输入本星期每天的最低气温:\n";
  for(Day i=Sun; i<=Sat; i=(Day)(i+1))
  {cin >> m[i]; min += m[i];}
  cout << "本星期平均最低温度为" << min/7;
  return 0;}
```

5. 编写一个函数，判断int型参数是否为回文数 12321

```
#include <iostream.h>
const int MAX=10;
int main()
{ int A[MAX], n;
  cout << "请输入一个数:";
  cin >> n; if(n==0) {cout << "是回文数"; return 0;}
  for(int i=0, temp=n; temp!=0; i++, temp/=10)
    A[i]=temp%10;
  //循环结束后i的值即为n的位数
  for(int j=0; j<i/2; j++)
  {if(A[j]!=A[i-j-1])
  {cout << "不是回文数"; return -1;}
  cout << "是回文数"; return 0;}
```

5. 编写函数：

```
#include <iostream>
using namespace std;
const int MAX=10;
bool is_huiwen(int n)
{ if(n==0) return true; int A[MAX];
  for(int i=0; n!=0; n/=10, i++)
  A[i]=n%10;
  for(int j=0; j<i/2; j++)
  { if(A[j]!=A[i-1-j]) return false;}
  return true;}
```

6. 编写一个函数 int_to_str，把int型数据改成字符串

```
#include <iostream>
using namespace std;
char *int_to_str(int n, char *p)
{ int i;  int temp=n;
  for(i=1; temp>9; i++, temp/=10);
  //上面求出n的位数i
  p=new char[i+1]; temp=n;
  for(int j=i-1; j>=0; j--)
  { p[j]=temp%10+'0'; temp/=10;}
  p[i]='\0';
  return p;}
```

7. 编写一个函数计算一元二次方程的根

要求：方程的系数和根由用户按值传递机制传递

```
#include <iostream>  #include <cmath>
using namespace std;  const double EPS=1e-7;
void f(double a, double b, double c,
       double &x1, double &x2)
{ int result;
 if(fabs(a)<EPS && fabs(b)<EPS
    && fabs(c)<EPS)  result=-3;
 else if (fabs(a)<EPS && fabs(b)<EPS)
   result=0;
 else if (fabs(a)<EPS)
   { x1=-c/b;  result=1; }
 else
 double delta=b*b-4*a*c; double temp=-b/2a;
  if (delta<-EPS)
  { x1=temp; x2=sqrt(-delta)/2a;
    result=-2;}
  else if (delta>EPS)
  { x1=temp; x2=temp; result=1;}
  else { x1=temp+sqrt(delta)/2a;
         x2=temp-sqrt(delta)/2a;
         result=2;}
 switch (result)
 { case -3: cout<<"方程有无穷多根"; break;
   case -2: cout<<"方程有两个复根,它们为"
     <<x1<<"+i"<<x2<<"和"<<x1<<"-i"
     x2; break;
   case -1: cout<<"方程有一个实根,"<<x1; break;
   case 0: cout<<"方程无根"; break;
   case 1: cout<<"方程有两个相同的根"
     <<x1; break;
   case 2: cout<<"方程有两个不同的实根:"
     <<x1<<"和"<<x2; break;}
}
```

8. 编写一个程序，从键盘输入一个字符串，分别统计其中的大写字母、小写字母和数字个数

```
#include <iostream.h>
int main()
{ char *p[80]; int i=0, j=0, k=0, n=0;
  cout<<"请输入一个字符串";
  cin>>p;
  for(n=0; p[n]!='\0'; n++)
  { if(p[n]>='A'&&p[n]<='Z') i++;
    else if(p[n]>='a'&&p[n]<='z') j++;
    else if(p[n]>='0'&&p[n]<='9') k++;}
  cout<<"大写字母有"<<i<<"个"<<endl;
  cout<<"小写字母有"<<j<<"个.\n";
  cout<<"数字有"<<k<<"个.\n";
  return 0;}
```

10.

```
void swap(int a[], int m, int n)
{int temp, i;
 for(i=0; i<m; i++)
 {temp=a[0];
  for(j=1; j<m+n; j++)
   a[j-1]=a[j];
   a[m+n-1]=temp;
}
```

11. 编写一个函数int squeeze(char s1[], const char s2[])，它从字符串s1中删除所有在s2里出现的字符。函数返回删除的字符个数。

```
int squeeze(char s1[], const char s2[])
{ int count=0, m=0;
  for(int i=0; s1[i]!='\0'; i++)

  for(int j=0; s2[j]!='\0'; j++)
```

Chapter 5. homework !new

No.

Date

5. bool is_hw(int n)

{ int a[10], i=0, j=0, m=n;

while (m>0)

{ a[i]= m%10; m= m/10; i++; }

m=0;

for(j=0; j<i; j++) m+= m*10+a[j];

if(m==n) return true;

else return false; }

检验 12. m=12. a[0]=2, m=1, i=1

a[1]=1, m=0, i=2, m=0,

m=2, m=20+1 =21, m≠n → false,

m=0时, 直接→ m==n 成立. true.

6. void int_to_str(int n)

{ char str[11], temp;

int i=0, j=0, m=n;

while (m>0)

{ str[i]= m%10+'0';

m/=10; i++; } str[i]='\0';

for(; j< i/2; j++)

{ temp=str[j]; str[j]=str[i-1-j]; str[i-1-j]=temp;}

}.

检验: n=10: str[0]='0' m=1, i=1.

str[1]='1', m=0, i=2, str[2]='\0'.

⇒ str[0]='1', str[1]='0', str[2]='\0'.

∴ str="10".

No.

Date

Chapter 6 homework 11)

14. Class Time

```
{ int hour,min,sec;
  public:
  Time ( ) { hour=min=sec=0; }
  Time(int h, int m, int s)
  { hour=h; min=m; sec=s; }
  void set (int h, int m, int s)
  { hour=h; min=m; sec=s; }
  void increment ( )
  { int temp=hour*3600+min*60+sec+1;
    hour=temp/3600;
    min=(temp%3600)/60;
    sec=temp%60; }
  void display ( )
  { cout<<"The time is:"<<hour<<':'<<
    min<<':'<<sec<<endl; }
  bool equal (Time &other_time)
  { return (hour==other_time.hour)&&(min==
    other_time.min)&&(sec=other_time.sec); }
  bool less_than (Time &other_time)
  { return (hour>other_time.hour)||(hour==
    other_time.hour)&&(min>other_time.min)||
    (hour==other_time.hour)&&(min==other_time.min)
    &&(sec>other_time.sec); }
  { if (hour>other_time.hour) return true;
  };
```

```
#include <iostream>
using namespace std;
class Time { ..... };
int main ( )
{ Time t1(5,2,16),t2;
  t2.set(1,5,7);
  t1.display(); t2.display();
  t1.increment(); t1.display();
  if (t1.equal(&t2))
  cout<<t1.display()<<"和"<<t2.display()<<
  "时间相同!"<<endl;
  else cout<<t1.display()<<"和"<<t2.display()
  <<"时间不同!"<<endl;
  if (t1.less_than(&t2))
  cout<<t1.display()<<"早于"<<t2.display();
  else cout<<t1.display()<<"不早于"...
  return 0; }
```

17. #include <iostream>

```
using namespace std;
struct Node { int content; Node *next; };
class set
{ Node *head;
  public:
  set ( ) {head=null;}
  set (const Set&S);
  { head=S.head; }
  ~set( ) {head=null;}
  bool is_empty( ) const
  { return (head==null); }
  int size( ) const
  { int i=0; Node *p=head;
    while (p!=null)
    {i++; p=p->next;} return i; }
  bool is_element (int e) const
  { Node *p=head;
    for (; p!=null; p=p->next)
    if (p->content==e) return true;
    return false; }
```

```
bool is_subset(const Set &s) const
{ if(this->isempty()) return true;
  else { Node *q=S.head;
         for( ; q!=null; q=q->next)
不用this可以吗? { if(!is_element(q->content))
             return false; }
         return true; } }
可以这样写 { Node *q=s.head;
         for( ; q!=null; q=q->next)
         { if(!is_element(q->content))
           return false; }
           return true; } }
bool is_equal(const Set &S) const
{ return (this->is_subset(S))||(S.is_subset(this)); }
void display() const
{ Node *p=head; int count=0;
 for( ; p!=null; p=p->next)
 { cout<<p->content<<','; count++;
  if(count%10==0) cout<<endl; } }
Set &insert(int e)
{ if(is_element(e)) return *this;
  else { Node *p=head; *q;
   q->content=e;
   for( ; p->next!=null; p=p->next);
   p->next=q; q->next=null;
   return *this; } }
可换位置 else
  else { Node *p=head->next; p->content=e;
         head=p; return *this; } }
Set &remove(int e)
{ if(!is_element(e)) return *this;
  else { Node *p1=head, *p2=null;
    for( ; p1!=null; p1=p1->next)
    { if(p1->content==e) break; p2=p1; }
    if(p1==head) { head=head->next;
                   delete p1; }
 确定p2!=null
    else { p2->next=p1->next; delete p1; }
    return *this; }
Set union(const Set &s) const
{ Set *U=new Set;
  Node *p=head;
  for( ; p!=null; p=p->next)
  (*U).insert(p->content);
  for(p=S.head; p!=null; p=p->next
    (*U).insert(p->content);
  return *U; }
Set intersection(const Set &S) const
{ Set *Inter;
  for(Node *p=head; p!=null; p=p->next)
  { if(S.is_element(p->content))
    (*Inter).insert(p->content) }
  return *Inter; }
Set difference(const Set &S) const
{ Set *differ;
  for(Node *p=head; p!=null; p=p->next)
  { if(!S.is_element(p->content))
    (*differ).insert(p->content) }
  return *differ; }
};
int main()
{ int n, i;
  Set A, B
  cout<<"请输入集合A的元素个数:"<<endl;
  cin>>n;
  cout<<"请输入集合A的元素:";
  for(int j=0; j<=n; j++)
  {cin>>i; A.insert(i); }
```

Chapter 6 homework (2)

(补充说明:

(1) 函数定在类定义外.

(2) 做后两个函数不能用 set* 类型, 因为一个指针没有初始化等导致用于在类外定义. 因此可以使用 Set 类型的对象.

```
Set Set::union2(const Set&s) const
{ Set u;
  Node *p=head;
  for(;p!=NULL;p=p->next)
    u.insert(p->content);
  for(p=s.head;p!=NULL;p=p->next)
    u.insert(p->content);
  return u;}
)
```

```
cout<<"请输入集合B的元素个数:"<<endl;
cin>>n;
cout<<"请输入集合B的元素:";
for(int k=0;k<n;k++)
{cin>>k; B.insert(k);}
if(A.is_empty) cout<<"A为空集";
else{cout<<"A的元素个数为"<<A.size()<<",其元素为"<<endl; A.display();}
if(B.---                    同上
Set C(B);
```

17

```
#include<iostream>
#include<cstdio>
using namespace std;
class Set
{ int *p, n;
public:
 Set() {p=NULL; n=0;}
 Set(const Set&s)                !!!
 { int *p = new int[s.n];
  for(int i=0; i<s.n, i++)
   p[i]=s.p[i];
                  n=s.n;}
 ~Set() { delete []p; p=NULL;}
 bool is_empty() const
  {return (n==0);}
 int size() const {return n;}
 bool is_element(int e) const;
 bool is_subset(const Set&S) const;
 bool is_equal(const Set&S) const;
 void display() const;
 Set& insert(int e);
 Set& remove(int e);
 Set union2(const Set&s) const;
 Set intersection(const Set&S) const;
 Set difference(const Set&S) const;
 Set& operator=(const Set&S) const
};
bool Set::is_element(int e) const
{for(int i=0; i<n; i++)
  if(p[i]==e) return true;
  return false;}
bool Set::is_subset(const Set&S)const
{for(int j=0; j<S.n; j++)
  if(!is_element(S.p[j])) return false;
  return true;}
```

```
bool Set::is_equal(const Set&S)const
{return (is_subset(S) && S.is_subset(*this));}
void Set::display() const
{for(int i=0; i<n; i++)
 cout<<p[i]<<'\t';}
Set& Set::insert(int e)
{int *p1=new int[n+1];
 for(int i=0; i<n; i++)
  p1[i]=p[i];
 p1[n]=e;
 delete []p;
 p=p1; n++; return *this;}
Set& Set::remove(int e)
{if(!is_element(e))
 {cout<<e<<"不在集合内元素"; return *this;}
 int *p1=new int[n-1];
 for(int i=0; p[i]!=e; i++)
 p1[i]=p[i];
 for(int j=i+1; j<n; j++)
 p1[j-1]=p[j];
 delete []p; p=p1; n--; return *this;}
Set Set::union2(const Set&S)const
{int m=0;
 for(int i=0; i<S.n; i++)
 if(!is_element(S.p[i])) m++;
 int *p1=new int[n+m];
 for(int j=0; j<n; j++)
 p1[j]=p[j]
 for(int k=0, count=0; k<S.n; k++)
 if(!is_element(S.p[k]))
 {p1[n+count]=S.p[k];
  count++;}}
Set Set::intersection(const Set&S)const
{int *p1=new int[n], count=0;
 for(int i=0; i<S.n; i++)
 if(is_element(S.p[i]))
 {p1[count]=S.p[i]; count++;}
 if(is_element(e))
 return *this}
Set Set::union2(const Set&S) const
{Set U(*this); for(int i=0;i<S.n;i++)
  U.insert(S.p[i]);
 return U;}
Set Set::intersection(const Set&S)const
{Set I;
 for(int i=0; i<S.n; i++)
 if(is_element(S.p[i]))
 I.insert(S.p[i]); return I;}
Set difference(const Set&S) const
{Set D;
 for(int i=0; i<n; i++)
 if(!S.is_element(p[i]))
 D.insert(p[i]); return D;}

int main()
{Set A[3]; int n;
 cout<<"请输入集合A[i]的元素个数:\n";
 cin>>n;
 cout<<"请输入集合A[i]元素:\n";
 for(int j=0; j<n; j++) {cin>>i;
 A.insert(i);}
 for()
 {cout<<"集合A[]";
 if(A[].is_empty()) cout<<"为空集";
 else {cout<<"不为空集元素个数为:"<<A[].size()
  <<"A中元素为:"; A[].display();}}
```

Chapter 6 homework 13)

```
cout<<"请输入要从A[0]中删除的元素";
cin>>i;  A[0].remove(i);
A[1]=A[0].union2(A[1]);
A[2]=A[0].intersection(A[1]);
A[3]=A[0].difference(A[1]);
for(j=0;j<3;j++)
```

不调用默认的赋值操作，逐个成员拷贝复制！

19. class A
```
{ static int *share;
  public:
    A(){ if(share==NULL) share=this;
         else this=share;}
};
int* A::share=NULL;
```

```
Set U;  //这时 U.p=NULL, U.n为0.
U=A[i].union2(A[j]);
```
return 操作时，创建一个临时对象，用此值给，这时返回的Set类型临时对象其指针和U指向同一个地方（逐个复制）返回后，U被析构函数 U.p=NULL.
因此返回的指针值为NULL，
因此，在有指针成员的类构成员时，必须注意！拷贝构造函数，析构函数，重载=运算必须一起全部写出来！
情况① 定义对象时，A a1; A a2(a1); 或A a2=a1; A a2=A(a1);
② 把对象作为值参数传给函数时
```
void f(A x); A a;
f(a);
```
③ 把对象作为返回值
```
A f()
{ A a;
  ...
  return a;
}
```
都用到了Copy构造函数与赋值操作，有指针类型成员时，很有可能会出现错误!!!

Chapter 10. Homework (1)

1. 编一个模拟程序，把一个文件中的内容复制到另一个新文件中去。

```
#include <iostream.h>
#include <fstream.h>
int main()
{ cout << "请输入要复制的文件，形如C:\\score.text.\n";
  char *p;
  while (cin>>p)
  { ifstream in_file(p, ios::in);
    if (!in_file)
      cerr << "地址无效！请重新输入地址或输入"END"结束.\n"; else break;
    if(strcmp(p,"END")) return 1;}
//循环结束时，文件打开成功或已退出程序.
  cout << "请输入要复制到的文件地址.\n";
  char *q;
  while (cin>>q)
  { ofstream out_file(q, ios::out);
    if (!out_file)
    cerr << "地址无效！请重新输入或"END"结束.\n";
    else break;}
//循环结束时，文件打开成功，或已退出程序.
  char a;
  while (!in_file.eof())
  { in_file >> a;
    out_file << a;}
  in_file.close();
  out_file.close();
  return 0;
}
```

2. 编写一个程序，统计一个文本文件的行数。

```
#include <iostream.h>
#include <istream.h>
#include <string.h>
int main()
{ cout << "请输入要统计行数的文本文件的地址.\n";
  char *p;
  while (cin>>p)
  { if strcmp(p,"END")
    { cout << "退出!" << endl; return 1;}
    ifstream in_file(p, ios::in);
    if(!in_file)
    cerr << "地址错误…… " << endl;
    else break;}
  int lin_num=1;
  char a; in_file.get(a);
  while (!in_file.eof())
  { if (a=='\n') lin_num++;
    in_file.get(a);}
  cout << "行数为" << lin_num;
  in_file.close();
  return 0;}
```

3.

```
#include <iostream.h>
#include <ostream.h>
#include <string.h>   #include <iomanip.h>
const int MAX_ID_LEN=10;
const int MAX_NAME_LEN=8;
const int MAX_ADD_LEN=50;
const int MAX_NUM_OF_TN=12;
class worker_inf
{ char id[MAX_ID_LEN+1];
  char name[MAX_NAME_LEN+1];
  char work_add[MAX_ADD_LEN+1];
```

Chapter 10 Homework (2)

```
    char tel_num[MAX_NUM_OF_INT];
    bool initialized;
public:
    worker_inf() { initialized = false; }
    bool data_is_OK() { return initialized; }
friend ostream& operator<<(ostream& out, worker_inf& x);
friend istream& operator>>(istream& in, worker_inf& x);
};
ostream& operator>>(istream& in, worker_inf& x)
{ if(&in==&cin)
  cout<<"请输入姓名,地址,电话号码,或以'E'结束\n";
  in>>setw(11)>>x.id;
  if(in.eof()||x.id[0]=='E')
  { x.initialized=false; return in; }
  in>>setw(9)>>x.name;
  in>>setw(51)>>x.work_add;
  in>>setw(8)>>x.tel_num;
  x.initialized=true;
  return in; }
ostream& operator<<(ostream& out, worker_inf& x)
{ out<<x.id<<'\t'<<x.name<<'\t'<<x.work_
  add<<'\t'<<x.tel_num<<endl; return out; }
int main()
{ cout<<"请输入要存储的地址.\n";
  char p[100];
  while(cin>>p)
  if(strcmp(p,"END")) return 1;
  ofstream out_file(p, ios::out);
  if(!out_file) cerr<<"  !\n";
  else break; }
  worker_inf worker;
  cin>>worker;
  while(worker.data_is_OK())
  { outfile<<worker; cin>>worker; }
  outfile.close();
  return 0; }
```

Date

1. 答!

```
#include<iostream>
#include<fstream>
using namespace std;
int main()
{ char p[100], q[100];
  cout<<"请输入要复制的文件\n";
  cin>>p;
  ifstream in_file(p, ios::in);
  if(!in_file)
  { cerr<<"找不到该文件"; return 1; }
  cout<<"请输入要复制到的文件\n";
  cin>>q;
  if(!strcmp(p,q)) return 1;
  ofstream out_file(q, ios::out);
  if(!out_file)
  { cerr<<"不能打开该文件"; return 1; }
  char ch; in_file.get(ch);
  while(!in_file.eof())
  { in_file.get(ch);
    out_file.put(ch); }
  in_file.close();
  out_file.close();
  return 0;
}
```

No.

Date

5.4. N键盘输入一个字符串，逆向输出

```
#include <iostream>
#include <cstring>
using namespace std; int main()
{ char A[100];
cout << "请输入一个长度小于100的字符串:" << endl;
cin >> A;
int i = strlen(A);
for (int j = i-1; j >= 0; j--)
cout << A[j];
return 0; }
```

另解:

```
int main()
{ const int MAX_NUM = 100;
  char str[MAX_NUM], temp;
  cin >> str;
  int len = strlen(str);
  for (int j=0, i=len-1; j < len/2; j++, i--)
  { temp = str[i];
    str[j] = str[j];
    str[j] = temp; }
  cout << str;
  return 0; }
```

5.5. 编写 str_to_int 字符串 -> 整型数.

```
#include <iostream>
(#include <cstring> )
using namespace std;
int str_to_int (char str[])
{ int str_int = str[0] - '0';
  for (int i=1; str[i] != '\0'; i++)
    str_int = str_int * 10 + (str[i] - '0');
  return str_int; }
```

(也可用求长度)

5.6. find_substr, 查找一个字符串在另一字符串中位置

```
int find_substr (char sub_str[], char str[])
{ int sub_str_len = strlen(sub_str),
    str_len = strlen(str);
  for (int i=0; i<=str_len-sub_str_len; i++)
  { int j=0;
    while (j<= sub_str_len && str[i+j]==sub_str
      [j]) j++;
    if (j== str_len) return i; }
  return -1; }
```

5.7. 输入一个 NXN 矩阵，输出其转置.

```
#include <iostream>
using namespace std;
int main ()
{ const int N = 5;  int temp;
  double A[N][N];
  for (int i=0; i<N; i++)
    for (int j=0; j<N; j++)
      cin >> A[i][j];
  for (i=1; i<N; i++)
    for (j=0; j<i; j++)
    { temp = A[i][j];
      A[i][j] = A[j][i];
      A[j][i] = temp; }
  for (i=0; i<N; i++)
  { for (j=0; j<N; j++)
      cout << A[i][j] << '\t';
    cout << endl; }
  return 0; }
```

(可加入一些 cout)

5.8 实现矩阵乘法 C=AxB.

```
#include <iostream.h>
int main()
{ const int m=5, n=3, f=4;
```

```
int A[m][n], B[n][l], C[m][l], i,j,k;
cout<<"请输入矩阵A:"<<endl;
for (i=0; i<m; i++)
  for(j=0; j<n; j++)
    cin >> A[i][j];
cout<<"请输入矩阵B:"<<endl;
for(i=0; i<n; i++)
  for(j=0; j<l; j++)
    cin >> B[i][j];
for(i=0; i<m; i++)
  for(j=0; j<l; j++)
  { C[i][j]=0;
    for(k=0; k<n; k++) C[i][j]+=
    A[i][k]*B[k][j]; }
for(i=0; i<m; i++)
 {for(j=0; j<l; j++) cout<<C[i][j]<<' ';
  cout<<endl; }
return 0; }
```

5.10. 从键盘输入n个整型数，从小到大排序，输出结果

```
#include <iostream>
using namespace std;
void sort(int A[], int n)
{ for(int i=0; i<n-1; i++)
    for(int j=0; j<n-i; j++)
    { if(A[j]>A[j+1])
      { int temp=A[j];
        A[j]=A[j+1];
        A[j+1]=A[j]; } } }
void display(int A[], int n)
{ for(int i=0; i<n; i++)
  {cout<<A[i]<<'\t';
  if(i%6==0) cout<<endl; }
int main()
{ const int N=100;
  int A[N], k;
  for(k=0; k<N; k++)
    cin>>A[k];
  sort(A,N);
  display(A,N);
  return 0; }
```

5.11 名称查找

```
#include <iostream>  #include <cstring>
using namespace std;
struct city_inf
{ char city_name[20];
  char city_numble[7];
  char city_tel[5]; };
int linear_search(char key[], city_inf
city[], int num)
{ for(int index=0; index<num; index++)
    if(strcmp(key, city[index].city_name)==0)
    break;
  if(index==num) return -1;
  else return index; }
int main()
{ const int numble=10; char str[20];
  city_inf city[numble];
  for(int i=0; i<numble; i++)
  { cin>>city[i].city_name>>city[i].city_numble
    >>city[i].city_tel; }
  cout<<"请输入所需查找的城市:"<<endl;
```

两种排序方法：选择排序，基本思想：从n个数中找出最大者，与第n个数交换，然后从n-1个数中找出最大者，与第n-1个数交换…

```
cin>>str;  int i=0;
i=linear_search(str, city, numble);
if(i==-1)  cout<<"error"<<endl;
else  cout<<city[i].city_numble<<endl;…
return 0;}
```

二分查找：首先要对它们排序，再二分查找.

```
//这里假设城市名从小到大已排好顺序.
int binary_search(char key[], city_inf city[],
{int first=0, last=num-1;          int num)
 while(first<=last)
 {int index=(first+last)/2;
  if(strcmp(key, city[index].city_name)==0)
  return index;
  else if(strcmp(key, city[index].city_name)>0)
  first=index+1;
  else last=index-1;}
  return -1;}
```

冒泡排序法：

```
void bubble_sort(int a[], int num)
{int j=-1;
 while(      num>0)
 {j=0;
  for(int i=0; i<=num-1; i++)
  if(a[i]>a[i+1])
  {temp=a[i];
   a[i]=a[i+1];
   a[i+1]=temp;}++j; if(j==0) break;
  num--;}}
```

功能：输入若干个数进行排序，以-1为结束标志，并从小到大输出这些数.

```
#include <iostream>
using namespace std;
const int N=10;  void sort(int p[],int n)
int main()
{int max_len=20;  int count=0, n;
 int *p=new int[max_len];  cin>>n;
 while(n!=-1)
 {if(count>=max_len)
  {max_len+=N;
   int *q=new int[max_len];
   for(int i=0; i<max_len; i++) q[i]=p[i];
   delete[]p;
   p=q;}
  p[count]=n;  count++;
  cin>>n;}
 sort(p, count);
 for(int j=0; j<count; j++)
 cout<<p[j]<<"\t";
 return 0;}
```

(*(p+i))

例1：根据输入要求执行一个函数表中定义的某个函数.

```
#include <cmath>
#include <iostream>
using namespace std;
typedef double (*FP)(double);
int main()
{double x, index;
 FP function_list[8]={sin, cos,
 tan, asin, acos, atan, log, log10};
do{cout<<"选输入所要计算的函数\n"
 <<"[0:sin, 1:cos, 2:tan, 3:asin, 4:acos,
  5: atan, 6: log, 7: log10)"<<endl;
 cin>>index;}
 while(index<0||index>7)
```

```
cout<<"请输入参数:";
cin>>x;
cout<<"结果为"<<function_list[index](x);
return 0;}
```
也可用(*function_list[index])(x);

用面向对象的方法实现类型"栈"类的类型及操作.

```
#include <iostream>
using namespace std; const int size_stack=100;
class stack
{ int top;
  int buffer[size_stack];
public:
  stack() {top=-1;}
  bool push(int i)
  { if(top==size_stack-1)
    {cout<<"stack is overflow.\n";
     return false;}
    top++;
    buffer[top]=i;
    return true;
  }
  bool pop(int &i)
  { if(top==-1)
    {cout<<"Stack is empty.\n";
     return false;}
     i=buffer[top];
     top--;
     return true;}
};
int main()
{ Stack st; // 自动调用 St.Stack() 进行初始化
  int x;
  st.push(12);
  st.push(8);
  st.pop(x);
  return 0;}
```

注:不能访问私有成员!

例:6-1 用链表实现栈类

```
#include <iostream>
#include <cstdio>
using namespace std;
class stack
{ struct Node{int content; Node* next;}*top;
 public: stack(){top=NULL;}
        bool push(int i);
        bool pop(int &i);};
bool stack::push(int i)
{ Node *p=new Node;
  if(p==NULL)
  {cout<<"Stack is overflow.\n";
   return false;}
  else{p->content=i; p->next=top;
       top=p; return true;}}
bool Stack::pop(int &i)
{ if(top==NULL)
  {cout<<"Stack is empty.\n";
   return false;}
  else{Node *p=top;
       i=p->content; top=top->next;
       delete p; return true;}}
```

表指表示不影响使用者.因为接口(public的成员函数)

例 6-2. 从键盘中输入一个日期值,判断是为闰年.

```
#include <iostream>
using namespace std;
```

No.

```
class Date
{ int year, month, day;
public:
Date() { year=month=day=0; }
Date(int y, int m, int d)
{ year=y; month=m; day=d; }
 void set(int y, int m, int d)
 { year=y; month=m; day=d; }
 bool is_leap_year()
 { return ((year%4==0) && (year%100!=0))
          || (year%400==0); }
 void print() const
 { cout << year << '.' << month << '.' << day; }
};
int main()
{ int y, m, d; Date some_day;
 cout << "请输入日期:年.月.日)" << endl;
 cin >> y >> m >> d;
 some_day.set(y, m, d);
 some_day.print();
 if(some_day.leap_year())
 cout << "是闰年。\n";
 else cout << "不是闰年。\n";
 return 0; }
```

例16.3 字符串类的定义(用成员函数实现基本操作)

```
#include <iostream>
#include <cstring>
#include <cstdio>
using namespace std;
class String
{ char *str;
public:
String() { str=NULL; }
String(char *p)
{ str = new char[strlen(p)+1];
  strcpy(str, p); }
~String { delete []str; str=NULL; }
int len()
{ return strlen(str); }
char &char_at(int i)
{ if(i<0 || i>=strlen(str))
 {cerr<<"超过字符串范围.\n"; exit(1);}
 else return str[i]; }
const char* get_str() { return str; }
String &copy(const char *p)
{ char *p1 = new char[strlen(p)+1];
  strcpy(p1, p);
  delete []str;
  str=p1;
  return *this; }
String &copy(const String &S)
{ return copy(S.str); }
String &append(const char *p)
{ char p1=new char[strlen(str)+strlen(p)+1];
  strcpy(p1, str);
  strcat(p1, p);
  delete []str;
  str=p1;
  return *this; }
String &append(const String &S)
{ return append(S.str); }
int compare(const char*p) { return strcmp
int compare(const String &S)   (str, p); }
 { return strcmp(str, S.str); }
};
```

```
int main ()
{ String S1, S2("abcd");
  S1.copy ("1234");
  S2.append (S1). append ("efg");
  for (int i=0; i<S2.len( ); i++)
    if (S2.char_at (i)>='a' && S2.char_at
      (i)<='z')
    S2.char_at (i) = S2.char_at(i)-'a'+'A';
  if (S2.compare ("ABCD1234EFG")==0)
  cout<<"OK\n";
  cout << S1. get_str() << S2. get_str();
  return 0; }
```

例6.5 用类来实现矩阵和向量类型并实现矩阵与向量相乘操作.

```
#include <iostream>
#include <cstdlib>
using namespace std;
class Matrix
{ int *p_data;  int row, col;
public:
  Matrix (int r, int c)
  { if (r<=0 || c<=0)
    {cerr <<"矩阵尺寸不合法! \n"; exit(-1);}
    row=r; col=c;
    p_data = new int[row*col]; }
  ~Matrix() { delete [] p_data; }
  //访问矩阵元素
  int &element (int i, int j)
  { if (i<0 || i>=row || j<0 || j>=col)
    {cerr<<"矩阵下标越界 \n"; exit(-1);}
    return p_data[col*i+j];
  }
  //访问矩阵元素(为常主对象提供)
  int element (int i, int j) Date const
  { if (i<0||i>=row || j<0 || j>=col)
    {cerr <<"矩阵下标越界"; exit(-1);}
    return *(p_data + i*col +j); }
  //获得矩阵的行数.
  int dimension_row ( ) const
  {return row; }
  int dimension_col ( ) const {return col; }
  //显示矩阵元素
  void display ( ) const
  { for (int i=0; i< row; i++)
    {for ( int j=0; j< col ; j++)
     cout <<*(p_data+i*col+j) << '\t';
     cout<<endl;} }
或 void display () const
    { int *p= p_data;
      for ( int i=0; i< row; i++)
       {for ( int j=0; j< col; j++)
          {cout <<*p << '\t'; p++}
        cout << endl;} }
};
class Vector
{ int *p_data;
  int num;
public:
  Vector (int n)
  { if( n<=0) {cerr<<"尺寸不合法! \n"; exit(-1);
    p_data= new int[n]; num=n;}
  ~Vector ( ) { delete [] p_data; }
  //访问向量元素
  int &element (int i)
  {if (i<0 || i>=num)
    {cerr<<"下标越界"; exit(-1); }
```

```
return *(p.data+i);}
int element(int i) const
{if(i<0||i>=num)
 {cerr<<"下标越界"; exit(-1);}
 return *(p_data+i);}
int dimension() const {return num;}
void display() const
{for(int i=0; i<num; i++)
 cout<<*(p+i)<<'\t'; cout<<endl;}
};
void multiply(const Matrix &m, const Vector
&v, Vector &r)
{if(m.dimension_col()!=v.dimension()||
   m.dimension_row()!=r.dimension())
{cerr<<"矩阵和向量尺寸不匹配.\n"; exit(-1);}
 for(int i=0; i<=m.dimension_row(); i++)
 {r.element[i]=0;
  for(int j=0; j<v.dimension(); j++)
  r.element[i]+=m.element(i,j)*v.element(j);
 }}
int main()
{int row, column;
 cout<<"请输入矩阵的行数,列数";
 cin>>row>>column; Matrix A(row,column);
 cout<<"请输入矩阵元素:\n";
 for(int i=0; i<row; i++)
  for(int j=0; j<column; j++)
  {cin>>A.element(i,j);}
 cout<<"请输入向量元素:\n"; Vector B(column);
  for(i=0; i<column; i++)
   cin>>B.element(i);
 Vector C(row);
 multiply(A,B,C);
 cout<<"矩阵:\n"; No.
 A.display(); cout<<"Date向量:\n";
 B.display(); cout<<"乘积为:\n";
 C.display(); return 0;}
```

说明: multiply 多次通过调用成员函数 element 访问 m, v, r 的元素. 每一次调用都要进行下标的合法性检查, 效率不高. → 作为友元!

```
class Vector;
class Matrix
{...
 friend void multiply(const Matrix &m,
 const Vector &v, Vector &r);
};
class Vector
{...
 friend void multiply(const Matrix &m,
 const Vector &v, Vector &r);
};
void multiply(const Matrix &m,
const Vector &v, Vector &r)
{if(m.col!=v.num||m.row!=r.num)
 {cerr<<"...."; exit(-1);}
int *p_m=m.p_data; *p_r=r.p_data; *p_v;
for(int i=0; i<m.row; i++)
 {*p_r=0; p_v=v.p_data;
  for(int j=0; j<m.col; j++)
  {*p_r+=(*p_m)*(*p_v);
   p_m++; p_v++;} p_r++;}
}
```

例5.1 输入/输出枚举类型的值

```
#include<iostream.h>
enum Day {Sun, Mon, Tue, Wed, Thu,
    Fri, Sat};
```

```
int main ()
{ Day d; int i;
  cin >> i;
  if (i<0 || i>7)
  {cerr << "Input Error! \n"; exit(-1);}
  d=(Day) i;
  switch (d)
  {case Sun: cout<<"Sunday"<<endl; break;
   case Mon: cout<<"Monday"<<endl; break;
   case Tue: cout<<"Tuesday"<<endl; break;
   case Wed:
   case Thu:
   case Fri:
   case Sat: cout<<"Saterday"<<endl;break;
  }
  return 0; }
```

第7章

例1. 实现复数运算及加、减、除.

```
class Complex
{ double real, imag;
public:
 Complex () { real=0; imag=0;}
 Complex (double r, double i) {real=r; imag=i;}
 ~Complex ();
 Complex operator + (Complex &S) const
 { Complex temp;
   temp.real = real + S.real;
   temp.imag = imag + S.imag;
 } return temp; // 调用3默认拷贝构造
 Complex operator - (const Complex &S) const;
 friend Complex operator * (const Complex &C1, const Complex &C2);
 friend Complex operator ÷ (const Complex &C1, const Complex &C2);
};
Complex Complex::operator - (const Complex &S)
{ Complex temp;
  temp.real = real - S.real;
  temp.imag = imag - S.imag;
  return temp;}
Complex operator * (const Complex &C1,
                    const Complex &C2)
{ Complex temp;
  temp.real = C1.real * C2.real - C1.imag * C2.imag;
  temp.imag = C1.real * C2.imag + C2.imag * C1.real;
  return temp;}
Complex operator ÷ (const Complex &C1,
                    const Complex &C2)
{ if((C2.real==0)&&(C2.imag==0))
  {cerr << "操作不合法!"; exit(-1);}
  Complex temp;
  double x = C2.real * C2.real + C2.imag * C2.imag;
  temp.real = (C1.real * C2.real + C1.imag * C2.imag)/x;
  temp.imag = (C1.imag * C2.real - C1.real * C2.imag)/x;
  return temp;
}
```

操作时,

```
Complex a(1,2), b, C(5,6);
b = a + C;  // 或 b = a.operator + (C);
b[1] = a - C;  // b[1] = a.operator - (C);
b[2] = a * C;  // b[2] = operator * (a, C);
b[3] = a ÷ C;  // b[3] = operator ÷ (a, C);
```

例7.3 重载操作符+，使其能实现两个复数的混合运算。

```
#include <iostream>
using namespace std;
class Complex
{ double real, imag;
public:
  Complex() { real=0; imag=0; }
  Complex(double x, double y) { real=x; imag=y; }
friend Complex operater + (double d, const Complex &s);
friend Complex operator + (const Complex &s, double d);
friend Complex operator + (const Complex &s, const Complex &c);
};
Complex operator + (double d, const Complex &s)
{ Complex temp;
  temp.real = d + s.real;
  temp.imag = imag;
  return temp; }
Complex operator + (const Complex &s, double d)
{ return Complex(s.real+d, s.imag); }
                    生成一个临时对象返回.
Complex operator + (const Complex &s, const Complex &c)
{ return Complex(s.real+c.real, s.imag+c.imag); }

int main()
{ Complex a(1,2), b(3,4), c1, c2, c3; double x=5;
  c1 = a+b;
  c2 = x+a;
  c3 = a+5.6; ... }
```

第一个 Complex operator + (double d, const Complex &s) 必须用全局函数来实现，因为第一个参数为double类型，而我们要重载操作符时，第一个必须是默认的 ~~const Complex *this~~ Complex *const this，指向类对象。

例：

```
class A
{ int x, y; char *p;
public:
  A() { x=y=0; p=NULL; }
  A(const char *str)
  { p=new char[strlen(str)+1];
    strcpy(p, str);
    x=y=0; }
  A& operator = (const A&s)
  { if (&s==this) return *this;
    delete[]p;
    p=new char[strlen(s.p)+1];
    strcpy(p, s.p);
    x=&s.x; y=&s.y;
    return *this; }
};
```

若没有重载操作符=的 A a("xyz"), b("abc"); a=b; 将就使a.p所原来指向的内容为"xyz"的内存无指针指向，内存漏，而a.p现指向"abc"，即 a.p == b.p 两个指向同一内存。

例10.6. 用I/O类库的读写机从键盘输入一批学生的成绩信息，以文本格式存入外部文件 Scores.text.

```
#include <iostream.h>
#include <fstream.h>
#include <iomanip>
const int MAX_NUM_OF_COURSES=30;
cont int MAX_ID_LEN=10;
const int MAX_NAME_LEN=8;
class StudentScores
{ int num_of_courses, score[MAX_NUM_OF_COURSES];
  char id[MAX_ID_LEN+1];
  char name[MAX_NAME_LEN+1];
  bool initialized;
public:
  StudentScore() {initialized=false;}
  bool data_is_OK() const {return initialized;}
friend ostream& operator << (ostream &out,
                             StudentScores &x);
friend instream & operator >> (istream &in,
                               StudentScores &x);
};
instream &operator >> (instream &in, StudentScores&x)
{ if (&in==&cin)
  cout<<"逐个输入学生的信息，选择学号为空时结束(以学号为'E'结束):\n";
  in>>setw(11)>>x.id; //x.id为字符串
  if (x.id[0]=='E' || in.eof()) //eof()检测文件
  {x.initialized=false;
   return in;}
  in>>setw(9)>>x.name;
  in>>x.num_of_courses;
  if(num_of_courses>MAX_NUM_OF_COURSES)
  {x.initialized=false;
   return in;}
  for(int i=0; i<x.num_of_courses; i++)
    in>>x.scores[i];
  x.initialized=true;
  return in;}
ostream &operator<<(ostream &out, StudentScores &x)
{ out<<x.id<<'\t'<<x.name<<'\t'
  <<x.num_of_courses<<'\t';
  for(int i=0; i<x.num_of_courses;i++)
  out<<x.scores[i]<<'\t';
  return out;}
int main()
{ ofstream out_file("d:\\scores.text", ios::out);
  if(!outfile) return -1;
  StudentScores st;
  cin>>st;
  while(st.data_is_OK())
  { out_file<<st<<endl; //往文件中写入学生信息
    cin>>st;
  }
  out_file.close();
  return 0;
}
```

对于 ostream 及 istream 重载的"<<"，">>"，其派生类 ofstream, ifstream 也可用.

例10.7. 从上例中输出的文件中读入数据，计算每个学生的成绩并输出到显示器.

```
#include <iostream.h>
#include <fstream.h>
#include <iomanip.h>
const int MAX_NUM_OF_COURSES=30;
```

```
const int MAX_ID_LEN=10;
const int MAX_NAME_LEN=8;
class StudentScores
{ int scores[MAX_NUM_OF_COURSES];
  int num_of_courses;
  char id[MAX_ID_LEN+1],
       name[MAX_NAME_LEN+1];
  bool initialized;
public:
  StudentScore() {initialized=false;}
  bool data_is_OK() {return initialized;}
  const char *get_id() const {return id;}
  const char *get_name() const {return name;}
  int average_score() const
  { double totle=0;
    for(int i=0;i<num_of_courses;i++)
    totle+=scores[i];
    return totle/num_of_courses;}
friend ostream& operator<<(ostream &out, StudentScores &x);
friend istream& operator>>(istream &in, StudentScores &x);
};
..... //与例10.6中的函数一样.
int main()
{ifstream in_file("d:\\scores.txt", ios::in);
 if(!in_file)
 {cerr<<"打开文件失败!\n"; return 1;}
  cout<<"姓名 性别 平均成绩:\n";
  StudentScores st;
  in_file>>st; // 从文件中读入数据
  while(st.data_is_OK())
  {cout<<st.id<<'\t'<<st.name<<'\t'
   <<st.average_score()<<endl;
   in_file>>st;
  }
  in_file.close(); return 0;}
```

No.

Date

Chapter 5 Homework (2)

```
if (S2[j]==S1[i])
{ count++;
  for (int k=i; S1[k]!='\0'; k++)
    S1[k] = S1[k+1]; i--; break; }
return count; }
```

12. int find_replace_str(char str[], const char find_str[], const char replace_str[])

```
{ int str_len = strlen(str),
    find_str_len = strlen(find_str),
    replace_str_len = strlen(replace_str); count=0;
  int m = find_str_len - replace_str_len, j=0;
  for (int i=0; i<=str_len-find_str_len; i++)
  { if (str[i]==find_str[0])
    {for (j=1; j<find_str_len; j++)
      if (str[i+j] != find_str[j]) break;
    if (j==find_str_len)
      { count++;
        if (m>=0)
        { for (int k=0; k<find_str_len; k++)
            str[i+k] = replace_str[k]; }
        if (m>0)
        { for (int k=0; k<replace_str_len; k++)
            str[i+k] = replace_str[k]
          for (int l=i+replace_str_len;
            str[l] = str[l+m]; }
```

16. 把链表中插入一个新结点的操作定义为一个函数

bool insert(Node *&h, int a, int pos);

h为表头指针，a为要插入的值，pos≥0

p=0，无条件，否则，在第pos个后面插入。

Node *&h 表示 Node* 类型的变量的引用类型，若只用 Node *h，表h本身的值不变，*h是可以改变的，而 Node *&h 类似于 int &a，a的值可变，这是 Node *h 可变，即h所指的内存空间可以更改，用引用类型还有个好处，即可以直接用h。(Node *)*h 则要用 *h 表示头指针。

```
struct Node;
bool insert(Node *&h, int a, int pos)
{ Node *p = new Node;
  p->content = a;
  if (h==NULL)
  { p->next=NULL; h=p; return true; }
  else if (pos==0)
  { p->next=h; h=p; return true; }
  else
  { Node *q=h; int j=1;
    while (j<pos && q->next!=NULL)
    {q=q->next; j++;}
    if (j==pos)  //说明第pos个结点存在.
    { p->next=q->next;
      q->next=p; return true; }
    else
    { cerr<<"不存在第"<<pos<<"个结点!\n";
      return false; }
  }
}
```

(pos>0)

17. bool remove(Node *&h, int &a, int pos)

```
{ if (h==NULL) return false;
  else if (pos==1)
```

```
{ Node *p=h; q=p->content;
  h=p->next; delete p; return true;}
else
{ int j=1; Node*p=h;
  while (j<pos-1 && p->next!=NULL)
  p=p->next;
  if (p->next!=NULL) //说明第pos个结点存在
  { Node *q=p->next;
    a=q->content;
    p->next=q->next;
    delete q; return true;}
  else return false;
}
```

构造函数 不能调用基类的。

派生类对象初始化由基类和派生类共同完成：基类的数据成员由基类的构造函数初始化。派生类的数据成员由派生类的构造函数初始化。当创建派生类对象时，派生类的构造函数在进入函数体前会调用基类的构造函数。→默认情况→缺省。否则必须成员初始化表中指出

~~XXX~~ 对于指针构造函数，派生类的默认 copy —— 会调用基类的copy构造函数。两个自定义的，默认情况下 → 默认构造函数。→ 显示指出不调用基类的指针构造函数。

~~XXX~~ C++为每个类定义了一个隐式的赋值操作符重载函数，行为：逐个成员进行赋值操作。每个成员：通常的赋值操作。成员对象：调用成员对象的赋值操作符=重载函数进行赋值。

对于包含成员对象的类来说，自己定义的赋值操作符重载函数不会自动地去调用成员对象的赋值操作符重载函数，必须显示指出。

=. 派生类不从基类继承赋值操作。若派生类未定义，则系统→默认。基类→调用基类的赋值操作符。派生类，成员逐个赋值。
自定义时，显示调用基类 =.

~~XXX~~ 类中若未定义 Copy 构造函数，则编译程序隐式提供。逐个成员 copy 初始化。对成员函数，调用成员对象类的 copy 构造函数。
自定义的 copy 构造函数不会缺省调用成员对象的 copy 构造函数，必须显示指出。（在成员初始化表若未指出，则表示调用成员的默认构造函数。

~~XXX~~ 赋值操作符只能通过非静态成员函数来重载，且不能被派生类继承。若派生类中未定义赋值操作符重载函数，则使用系统

第八章 继承——派生类

No.

Date

继承机制——软件复用的解决方法

在定义一个新的类时，或把一个或多个已有类的功能合并包含进来，也可再给出新功能的定义或对已有类的某些功能主新定义。

继承不需对现已有代码，基于目标代码的复用机制

8.1 基类，派生类

+派生类拥有基类的所有特征，并可定义新特征或主定义。

继承机制作用：①软件复用 ②对事物的进行分类（一般与特殊） ③支持软件的增量开发 ④对抽象进行组合

*表示关系：一种公与整体之间的关系

以往代码复用的简陋实现，聚集比继承也，可通过继承与封装的方面。

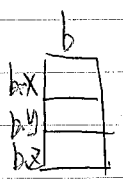
8.2 单继承

*定义：派生类只能有才直接基类。

```
class A: public B
{ ... };   private / protected.
```

继承方式可省略，默认为private. 用于说明派生类的实例（对象）及派生类的派生类对该派生类与基类成员的访问控制。

```
class A                    class B: public A
{ int x, y;                { int z;
public:                    public:
    void f();                  void h();
    void g();              };
};
```




```
B b; b.f(); b.g(); b.h();
```

(1) 派生类B拥有基类A的所有成员（除了构造函数和析构函数操作符外）也有其自身的成员

(2) 派生类的成员使用基中对基类的某成员进行了重定义，则派生类中对该成员的访问当缺省时对派生类中重新定义的成员。（正原则）

```
class B: public A
{ int z;
public:
    void f();
    void h() { f(); g(); }    // B中f.
};
```

则使用时，B b; b.f(); // 访问B类中的f

b.g(); // A中g();

b.h(); // B中的f(), A中的g();

(3) 派生类成员函数不可直接或简介访问基类中的私有成员，基类的隐藏。

```
class B: public A
{ int z;
public:
    void f();                          B类中的f.
    void h() { f(); A::f(); }
};
B b; b.f(); b.h(); b.A::f();
```

*即使派生类中定义与基类同名，但在派生类中同名成员也将作用域也不直接可用

```
class B: public A
{ int z;
public:
    void f(int);                  error      ok
    void h() { f(1); f(); A::f(); }
};
B b; b.f(1); b.f(); b.A::f();
```

(4) 定义派生类时一般不对从基类继承的成员定义。

(5) 基在派生类中要访问或调用基类的友元，非派生类的友元。基类为其一个类的友元，未说明时，派生类不为其友元。

(6) 派生类不能直接访问基类的私有成员！通过非私有成员访问

8.2.2 在派生类中对基类成员的访问

一个类有两种用户：实例用户，派生类。

一个类的private成员在该类外界使用后，在派生类中也不可使用

protected的成员，可被派生类使用，但不可被基类的实例用户使用

*软件复用：聚集✓，为类的有才接口，实例用户接口，继承功能可用客观实现，安全性好，继承更宜于实现可类型

多态作用①一个类型的操作中适合基类型，对派生
No.
型的对象可赋给或作为出现参数传给基类型参数
Date
②发给基类对象的消息也能发给派生类对象
③基类的对象、指针可被给派生类对象
A和B的基类，于为A类或B类的派生类，多态类，A中有
A a, *p; B b, *q;
b.f1(); a=b; //用b去给a的状态.
p=&b; 但 b=a; q=&a; //error.
8.2.3. 继承方式.
派生类有基类所有成员，基类的成员对派生类的
用户具有的访问控制
public继承方式可实现多态，其他代码上的继承
×继承方式的访问控制 除private成员其他√.

```
class A
{public:
  void f1();
  void f2();
  void f3();
protected:
  void g1();
  void g2();
  void g3();
};
```

```
class B: private A
{public:
  A::f1;
  A::g1;
protected:
  A::f2;
  A::g2;
private:
  A::f3;
  A::g3;
};
```

B b; b.f1(); b.g1(); √.
b.f2(); //error.
注意问题 P57.
8.2.4. 派生类对象的初始化和赋值操作
初始化：基类、派生类共同完成
一个类中既有基类又有成员对象类，则在创建该类对
象时，该类的构造函数先调用基类的、成员对象
自己的构造函数.
析构构造函数!
派生类不从基类中继承"="，隐式（系统）对基
类成员调用基类的赋值操作符，派生类通过赋值
若在派生类中重载"="，显示.

No.

1. for (char a='A'; a<='Z'; a++) cout<<a;

2. char A[10]={'0','1','2','3','4','5','6','7','8','9'};
   char A[]="0123456789";

3. char ch[10]; or char ch[10]="abcd";
   strcpy(ch, "abcd");
   若用 char ch[10]; ch="abcd"
   不报错时，否则，出错.

4. int a[3][3]={{0,2,1},
   {1,0,2},
   {1,2,0}};
   △计算数字. 重复计算少. 循环次少.

5. int hw(int n)
   { int m=0; //用来倒过来写num)
   for(int k=n; k>0; k/=10) m=m*10+k%10;
   if(m==n) return 1;
   else return 0;
   }

6. int main()
   { double a=0, b=0; int i=0;
   while((b=1/(2*i+1))>=1e-8)
   { if(i%2==0) a+=b;
   else a-=b;
   }
   return a;
   }

7. 求最大公约数.
   int main()
   {int c=(a<b)?a:b;
   while(b%c!=0||a%c!=0) c--;
   cout<<c; return 0;}
   利用辗转相除法.

8. 宏定义都加上括号
   #define SWAP(x,y) {(x)=(x)+(y); (y)=(x)-(y); (x)=(x)-(y)}

9. double 型比较
   极限不用 a==0. 可写 fabs(a)<1e-8;

10. int main()
    { int n, n3, sum=0, i=1, j;
    cout<<"请···比较:";
    cin>>n;
    n3=n*n*n;
    while(sum!=n3)
    { i+=2; for(sum=0, j=i; j<=i+2*(n-1); j+=2) sum+=j;
    }
    cout<<n<<'=';
    for(j=i; j<i+2*(n-1); j+=2)
    cout<<j<<"+";
    cout<<j<<endl;
    return 0;}

11. 集合!!! Vcampers Set.

No.

Date

1. C++的源程序文件扩展名为 .cpp .h

2. 编译检查语法错误.

3. 编译程序将C++源程序编译成目标代码.

4. 连接程序将编译好后目标程序和库文件连接成可执行文件. float 4字节.

5. int表示范围与2个字节. long则为4字节. unsigned int 2字节. short 2字节. double 8字节.

6. char s='\092'; 说明不合法, s的值不确定. 字符常量允许有两种形式表示'a'和以'\'开头的字符序列——转义字符序列. 表示以8进制, 不能包含数字9.

八进制: '\ddd', 十六进制: '\xhh'

7. 逻辑值"真"用非0的数表示? 0表示假. true. false

8. 条件表达式(M)?(a++):(a--)则表达式M和(M!=0)等价

9. if后面的判断表达式不限于比较关系表达式或逻辑表达式, 只要表达式的值为0或非0即可, ∴if后的表达式可以为任意类型的表达式.

10. "!"的运算符优先级高于%运算符. ∴!x%2的运算结果都为0. !x%2!=0 永远为"false".

!(x%2!=0) 可用以表示判断奇偶性

11. 10>x>10 的表达合法的, 编译时不会报错. 但若x=50时. 10>50的结果为0. 0>10表示结果为true. 显然不合逻辑.

12. 用逗号运算符将多个表达式连接起来形成逗号表达式. 表达式由左至右, 并以最右边表达式的结果作为整个逗号表达式的结果.

如(a=a+1, b+a, b+1)为一个逗号表达式. 它的值为最后一个表达式b+1的结果.

13. 赋值号的左边只能为一个变量, 不能为一个表达式. 如a=b=c+2; //OK. a=b, b=c+2.

a=a+1=c; ⇒ a=a+1 a+1=c; 不对.

14. a=int(i)? 对. a=(int)i;

15. k=32764;

do{k++;k++;}while(k>0);

不是死循环. 因为计算机系统硬件所限, k值不断个加, 自然溢出, 超出计算机系统所允许的整数范围. 从而造成k为负值或0. 从而结束循环.

16. C++采用 if(s1)if(s2)else<s2> else总与前面最近的, 没有else总和的if配对

if(x!=0)
if(x>0) y=1;
else y=-1;
else y=0;

$$y=\begin{cases}1, & x>0\\ 0, & x=0\\ -1, & x<0\end{cases}$$

17. char s[]="china"; char *p; p=s;

则s数组长度为6. 个所指向的字符串长度为5. p和s[0]相等. p存放的是"china"的首地址.

18. 为查找字符串sub是否为字符串st的子串. 若是, 则输出在st中出现的下标, 否则输出-1.

```
#include<iostream>
int main()
{ int i,j,k
  char st[100], sub[50];
  cout<<"Please Enter a string:";
  cin>>st;
  cout<<"Please Enter a sub string:";
  cin>>sub;
  for(i=0; st[i]!='\0'; i++)
  { for(k=0, j=i; st[j]==sub[k]&&
        sub[k]!='\0'; k++, j++);
    if(sub[k]=='\0')
    {cout<<i; break;}
  }
  if(st[i]=='\0')
  cout<<-1;
  return 0;}
```

19. 下列程序从键盘输入字符串给字符数组 s[] 中删除指定的字符后输出，请在横线处填入正确内容。

```
#include<iostream.h>
int main()
{ char s[80], c; int i,j;
  cout<<"Enter a string:";
  cin>>s;
  cout<<"Enter a character:";
  cin>>c;
  for(i=0; s[i]!='\0'; i++)
    if(s[i]!=c) s[j++]=s[i];
  s[j]='\0';
  cout<<s;
  return 0;
}
```

20. 以下 0≤i<10，C 则对数组元素的错误引用。

```
int a[]={1,2,3,4,5,6,7,8,9,10}, *p, i;
p=a;
```

A. *(a+i)　B. a[i]　C. p+i ← 地址　D. a[p-a]　E. *(&a[i])

p-a=0，因此 a[p-a] 表示 a[0]。

21. 若有以下说明和语句，0≤i<10，则 D 为对数组元素地址的正确表示。

```
int a[]={1,2,3,4,5,6,7,8,9,10}, *p, i; p=a;
```

A. &(a+1)　B. a++　C. &p　D. &p[i]

A. C. a, p 都表示 a[0] 首地址。

B. a 代表数组首地址，为一个地址常量，它的值在程序运行期间不能被改变。a++ 无法实现。

22. 指针数组 vs 指向一维数组的指针

```
int (*p)[3];
```

int *p; 是一个指向整型变量的指针。

相当于 int (*p);

则 int (*p)[3]; 即表示一个指向个有三个整型变量的一维数组的指针。

若 int *p[3]; 则 p[3] 为一个数组，*p[3] ⇔ *(p[3]) 每个元素都为一个指针。

指针数组 int *p[3] ⇔ (int*)[3]

为 int a[4][10], *p, *p[4], (*p)[10];

C++ 的编译系统对二维数组 a 进行处理时，把 a 看作由 a[0], a[1], a[2], a[3] 这 4 个元素组成的一维数组构成，其中每个元素又分别包含 10 个整型元素的一维数组。若一个整型占两个字节，则 a 为一个具有 4 个元素的一维数组名，其中每个元素的长度为 20 个字节。a[1] 则是 4 个一维数组，每个元素为整数。a 与 a[1] 为基类型不同的指针。

p=a[1]; p[1]=a[1]; p=&a[0][3]; p=a; 都是合法的。p=a; ✗

24. 若有说明：int a[4][10]; 则 ____ 为对数组元素 a[i][j] 的错误引用。

A. *(&a[0][0]+10*i+j)　B. *(a[i]+j)

C. *(*(a+i)+j)　D. *(a+i)[j]

a[i][j] 的地址可表示为 &a[0][0]+10*i+j;

a[i]+j 与 *(a+i)+j 表示相同。

a[i] 与 *(a+i) 都表示 a[i][0] … a[i][9] 一维数组的首地址，a[i]+j 则为其中第 j 个的地址。

注意：int *p[10], a[i]; 则 p[i] 表示 *(p+i)

则以上 (a+i)[j] 表示 *(a+i+j)

25. int a[3]={0}, *p[3], **pp, i;

for(i=0; i<3; i++) p[i]=&a[i];

pp=p;

则以下 ____ 为对 a 数组元素的错误引用。

A. a[1]　B. p[2][3]　C. a[i]　D. pp[0][1]

A. 下标越界。a[0], a[1], …, a[9]

p[0]=&a[0], p[1]=&a[1], p[2]=&a[8]

p[2][3] 和 *(p[2]+3), *(*(p+2)+3) 等价。

pp 表示一个指向整型指针的指针变量

pp[0][1] 等价于 *(pp[0]+1), *(p[0]+1) *(*(pp+0)+1)

pp[0] 表示 p[0]，即 *(pp+0)，为一个指针。

26. 下不正确的字符串赋值或赋初值方式（C）

A. char *str; str="strong";

B. char str[7]={'s','t','r','o','n','g'};

C. char str1[10]; str1="strong";

D. char str1[]="strong", str2[]="12345678";
strcpy(str2,str1);

A中定义了一个字符指针，赋值语句str="strong"；把一个存放字符串常量的无名存储区的首地址赋给了str，使str指向字符串的第一个字符。

B. 单个赋值，个数少于数组长度，后边补'\0'

C. 定义了一个含10个元素的字符型一维数组str1，str1为一个数组名，为一个地址常量，不允许整数赋值类似于 int a[10], *p=a; p++; //ok. a++; //error.

27. char *sp="\t\v\\\0will\n";
cout<<strlen(sp); 字符串结果为3.
\t, \v, \\, \n 都是一个字符，转义字符。

28. char *sp="\x69\082\n"; cout<<strlen(sp);
字符串结果为1. 在C++中，以一个反斜杠为转义字符，\后可跟1至3位的3进制数或跟字母x及1至2位的十六进制数，以此来代表一个特定的字符。
\x69 是用十六进制数表示的一个正确的ASCII码字符
但 \082. 不是八进制（八有8），则系统把 \082 分析为'\0', '8', '2' 3个字符。

29. 若对函数类型未加显示说明，则函数的隐含类型为 int 类型。

30. 以下函数调用语句中含有 3 个实参，
f((v1,v2),(v3,v4,v5),v6);
函数调用的一般形式为：函数名(实参1, 实参2, …)，实参可以为常量，变量和表达式。(v1,v2)为逗号表达式

31. 以下程序调用 getsum() 函数，把指针 p1 和 p2 所指存储单元中的数相加，其和放入指针 S 所指存储单元中，_____ 能正确执行以上功能的程序

A. #include <iostream.h>
void getsum(int *p1, int *p2, int *S)
{ int t; t=*p1+*p2; S=&t; }

B. void getsum(int *p1, int *p2, int S)
{ S=*p1+*p2; }

C. void getsum(int *p1, int *p2, int *S)
{ *S=new int;
*S=*p1+*p2; }
void main()
{int a,b,*p1=&a,*p2=&b,*S;
get(p1,p2,S);}

D. void getsum(int *p1, int *p2, int *S)
{ *S=*p1+*p2; }.

A. t为函数内局部变量，在getsum函数执行完后已不存在，从而 S=&t 没有意义。

C. S 为指向指针的指针变量

32. ____ 为main()函数参数的正确表述形式

A. main(int argc, char *argv[])

B. main(ac, av) int ac; char **av;

C. main(c, v) int c; char *v[];

D. main(argc, argv) int argc; char argv[];

main()函数的参数通常有两个，前者为整型，后者为字符型指针数组。**av 与 *av[] 等价

33. 若有以下调用语句，则不正确的fun()函数的首部为 _____

void main()
{… int a[50], n; …
fun(n, &a[9]); …}

A. void fun(…

B. void fun(int S, int h[4][]);

C. void fun(int *p, int *S);

D. Void fun(int n, int t);

为一个地址。

为一个指针变量。

35. 下程序统计有不及格(小于60)成绩的人数，并输出不及格成绩的学生的门成绩。学生数SIZE，每位学生4门考试成绩，指针型函数search()的形参为指向含有4个元素的一维数组的指针，函数检查某学生成绩，若有不及格，返回指向该生第一门成绩的指针，否则指向下个学生第1门成绩的指针。

```
#include <iostream.h>   #define SIZE 40
int *search (int (*pr)[4])
{ int i, *pc;
  pc = *(pr+1);
  for( i=0; i<4; i++)
     if (**(pr+i) < 60)
          { pc = *pr; break;}
  return pc; }
void main()
{ int i,j,m=0, *p, score[size][4];
  for( i=0; i<SIZE; i++)
  { cout << i << ":\n";
     for ( j=0; j<4; j++)
        cin >> score[i][j];
  }
  for ( i=0; i<SIZE; i++)
  { p=search(score+i);
     if (p==score[i]    //或 p=*(score+i)
     { m++;  cout<<"No."<<i<<"Score:";
       for( j=0; j<4; j++)
       cout<< score[i][j];
       cout<<endl;
     }
  }
  cout<<m;
}
```

score+i：指向第i行数组元素的首地址

pr+1：指向下一个一维数组的首地址

int (*pr)[4]：pr为一个指向一个一维数组的指针

*(score+i)：第i行第0列元素的指针

*pr：指向一维数组第0个位置的指针

*(pr+1)：指向下一个一维数组第0个位置的指针

*(pr+0)+i：指向下一个一维数组第i个位置的指针

即*(pr+1)+i)。

p+j：p所指元素向后j个元素的地址。

*(p+j)；

36.
```
void swap( int *p1, int *p2)
  { int *t;
    t=new int ____;
    *t=*p1; *p1=*p2; *p2=*t;
    delete t ____; }
```

局部指针t为一个指针变量，应使它指向一个具体的整型存储单元的地址为两数交换时的中间单元。

37.
```
#include <iostream.h>
#define M 3
#define N M+1
#define NN N*N/2
void main()
{ cout << NN; cout << 5*NN; }
```

输出第1个值为____ 第2个值为____

程序中定义3个不带参的宏名M、N、NN。在进行宏展开时，只需将宏名用宏定义中的字串按原样替换即可。

∴ NN = M+1*M+1/2 = 3+1*3+1/2 = 6.

5*NN = 5*N*N/2 = 5*M+1*M*1/2 =

关于同时对角化、同时标准化问题 (1)

No.

同时对角化、上三角化、标准化问题，常与矩阵乘法的可交换性有关。常用的方法是将问题归纳化，降阶从特征向量、不变子空间等，再用数学归纳法。

例1：A, B 为数域 F 上 n 阶矩阵，且 $AB=BA$。若 A, B 特征值都在 F 中，求证：存在 F 上可逆矩阵 P，使 $P^{-1}AP$，$P^{-1}BP$ 都是上三角阵。

例2：若复数矩阵 A, B 可交换即 $AB=BA$，则 A, B 至少有一个相同的特征向量。

证：取 $V=\mathbb{C}^n$，取定 V 的一组基 $e_1, \cdots, e_n$，φ, ψ 在 $e_1, \cdots, e_n$ 下对应矩阵分别为 A, B。则 $\varphi\psi=\psi\varphi$。

$\because \varphi$ 为 $\mathbb{C}$ 上的线性变换，必有特征值，设 λ_0 为 φ 的一个特征值，则记 λ_0 的特征子空间 $V_0=\{x \mid \varphi x=\lambda_0 x\}$。

则 $\varphi\psi x=\lambda_0\psi x=\psi\varphi x_0$。$\therefore \psi x_0 \in V_0$。$\therefore V_0$ 为 ψ 的不变子空间。将 ψ 限制在 V_0 上，分别记为 ψ_0, φ_0。则 ψ_0 在 V_0 上必有特征值 $\mu_0 \in \mathbb{C}$，对应特征向量 $x_0 \in V_0$。$\varphi_0(\mu_0)=\mu_0 x_0=\psi x_0$。

即 $\psi x=\lambda_0 x_0$。$\therefore x_0$ 即为 A, B 公共的特征向量。

例3：复矩阵 $\{A_i \mid i=1, \cdots, m\}$ 两两可交换，则它们至少有一个相同的特征向量。

证：V_1 为 A_1 的某个特征子空间，由 $A_1A_2=A_2A_1$ 可知，V_1 为 A_2 的不变子空间。将 A_2 限制在 V_1 上，则有 A_2 的不变子空间 $V_2 \subseteq V_1$。再由 $A_3A_2=A_2A_3$，可知，V_2 为 A_3 的不变子空间。一直下去，最后得到 V_{m-1} 为 A_{m-1} 限制在 V_{m-1} 上的特征子空间，$V_{m-1} \neq 0$。$\therefore$ 其中的非零向量即为 A_i 的公共的特征向量。

推广：设 A, B 为数域 F 上的矩阵且 $AB=BA$，A, B 特征值都 $\in F$，则 A, B 至少有一个相同的特征向量。

证：A 中 B 中 $\lambda_0 \in F$ 为 A 的一个特征值，V_0 为 A 的特征子空间，则 $\forall x_0 \in V_0$ 为 ψ 的不变子空间。

$\psi|_{V_0}=\psi_0$。则 ψ_0 的特征值必为 ψ 的特征值，从而属于 F。$\therefore \exists \mu_0 \in F$，$\psi_0 x_0=\mu_0 x_0$，$\lambda_0 \in V_0$，$x_0 \neq 0$。

即 $\psi x_0=\lambda_0 x_0$。$\therefore x_0$ 即为 A, B 公共的特征向量。

推广2：[标题为] 设 F 上矩阵且两两可换，A_i 的特征值都属于 F，则 A_i 至少有一个共同的特征向量。

证例1：用数学归纳法对阶数进行归纳。

当 $n=1$ 时，显然成立。假设 $n \leq k-1$ 时，结论成立，考虑 $n=k$ 的情况。($k \geq 2$)

$AB=BA$，且其特征值在 F 中，$\therefore A, B$ 至少有一个相同的特征向量设为 x_0。($x_0 \neq 0$)

则 $Ax_0=\lambda_0 x_0$，$Bx_0=\mu_0 x_0$。将 x_0 扩张为 V 的一组基，则 $A(x_0, e_2, \cdots, e_n)=(x_0, e_2, \cdots, e_n)\begin{pmatrix}\lambda_0 & * \\ 0 & P_{22}\end{pmatrix}$

$B(x_0, e_2, \cdots, e_n)=(x_0, e_2, \cdots, e_n)\begin{pmatrix}\mu_0 & * \\ 0 & Q_{22}\end{pmatrix}$，记 $(x_0, e_2, \cdots, e_n)=P_1$，则 $P_1^{-1}AP_1=\begin{pmatrix}\lambda_0 & * \\ 0 & A_{22}\end{pmatrix}$，$P_1^{-1}BP_1=\begin{pmatrix}\mu_0 & * \\ 0 & B_{22}\end{pmatrix}$

$\because AB=BA$，$\therefore \begin{pmatrix}\lambda_0 & * \\ 0 & B_{22}\end{pmatrix}\begin{pmatrix}\mu_0 & * \\ 0 & A_{22}\end{pmatrix}=\begin{pmatrix}\mu_0 & * \\ 0 & A_{22}\end{pmatrix}\begin{pmatrix}\lambda_0 & * \\ 0 & B_{22}\end{pmatrix}$，$\therefore B_{22}A_{22}=A_{22}B_{22}$

$\therefore$ 由假设 $\exists Q_1$ 可逆，使 $Q_1^{-1}A_{22}Q_1$，$Q_1^{-1}B_{22}Q_1$ 为上三角阵。

则记 $Q=\begin{pmatrix}1 & 0 \\ 0 & Q_1\end{pmatrix}$，则 Q 可逆，且记 $P=P_1Q$，则 $P^{-1}AP=Q^{-1}P_1^{-1}AP_1Q=\begin{pmatrix}1 & 0 \\ 0 & Q^{-1}\end{pmatrix}\begin{pmatrix}\lambda_0 & * \\ 0 & A_{22}\end{pmatrix}\begin{pmatrix}1 & 0 \\ 0 & Q_1\end{pmatrix}$

$=\begin{pmatrix}\lambda_0 & * \\ 0 & Q_1^{-1}A_{22}\end{pmatrix}\begin{pmatrix}1 & 0 \\ 0 & Q_1\end{pmatrix}=\begin{pmatrix}\lambda_0 & * \\ 0 & Q_1^{-1}A_{22}Q_1\end{pmatrix}$ 上三角阵。同理 $P^{-1}BP$ 也为上三角阵。

推广：当 $\{A_i \mid i=1, \cdots, m\}$ 两两可交换，特征值都属于数域 F，则 $\exists$ 可逆阵 P，使 $P^{-1}A_iP$ 同时为上三角阵。

例4：设 n 阶矩阵 $\{A_i \mid i=1, \cdots, m\}$ 两两可交换且都可对角化，证：存在可逆阵 P，使 $P^{-1}A_iP$ 同时为对

角阵

证：对阶数用归纳法：当n=1时，结论显然。假设当n≤k-1时，(k≥2)结论成立。则n=k时

将A_i视作V上的线性变换。$\lambda_1,\cdots,\lambda_t$为$A_i$的不同的特征值，$V_1,\cdots,V_t$为对应的特征子空间。

则由A_i可对角化可知 $V=V_1\oplus\cdots\oplus V_t$。$A_iA_j=A_jA_i$，$\therefore V_i$为$A_j$的不变子空间。

$\therefore$取V_i($i=1,\cdots,t$)的一组基，依次排列为V的一组基。则 $A_i(e_1,\cdots,e_n)=(e_1,\cdots,e_n)\begin{pmatrix}\lambda_1 I_{r_1}&&\\&\ddots&\\&&\lambda_t I_{r_t}\end{pmatrix}$

$A_j(e_1,\cdots,e_n)=(e_1,\cdots,e_n)\begin{pmatrix}A_{j1}&&\\&A_{j2}&\\&&\ddots&\\&&&A_{jt}\end{pmatrix}$ 即 $P^{-1}A_iP=\begin{pmatrix}A_{i1}&&\\&A_{i2}&\\&&\ddots&\\&&&A_{it}\end{pmatrix}$.

(这里若$t=1$，则$A_i=\lambda_i I_n$ 这样就只需考虑$A_2,\cdots,A_k$，若每个A_i的特征子空间都只有本V 则$A_i=\lambda_i I$ 显然结论成立。否则，不妨设A_1的特征子空间有多于1个，即$t\geq 2$)

A_i可对角化，则A_{ij}($i=1,\cdots,t$)也可对角化。再由A_i的不交换性可知$\{A_{ij}:i=1,\cdots,m\}$ $j=1,\cdots,t$也两两可交换，由归纳假设知，$\exists P_j$，$j=1,\cdots,t$，满足$P_j^{-1}A_{ij}P_j$均对角阵。$i=1,\cdots,m$

记$Q=\begin{pmatrix}P_1&&\\&P_2&\\&&\ddots&\\&&&P_t\end{pmatrix}$ 则$(PQ)^{-1}P^{-1}A_iPQ$为对角阵

例5：A为n阶正定阵，B为同阶实对称阵，则必存在可逆阵C，使$C'AC=I_n$，$C'BC=\mathrm{diag}\{\lambda_1,\lambda_2\cdots\lambda_n\}$，其中$\{\lambda_i\}$为矩阵$A^{-1}B$的特征值。

证，A为正定阵，则$\exists$可逆阵C $C^HAC=I_n$ 而C^HBC为对称阵，则$\exists$正交阵P，使

$P^{-1}C^HBCP$为对角阵，且对角线元素为C^HBC的特征值。而$P^{-1}C^HACC^{-1}=I_n$.

记$C^{-1}CP=D$，则D可逆，且$D'AD=I_n$，$D'B_2D=\mathrm{diag}\{\lambda_1\cdots\lambda_n\}$ $C^{-1}=C^HA^H$

而$C^HBC=C^{-1}A^{-1}BC$，$\therefore\lambda_1\cdots\lambda_n$为$A^{-1}B$的特征值。

例6：设φ,ψ为n维内积空间V上的两个自伴随变换，求证：这两个线性变换在V中存在由公共特征向量组成的标准正交基的充要条件为$\varphi\psi=\psi\varphi$.

证：$\Rightarrow$设这组标基为$e_1\cdots e_n$，则$\varphi e_i=\lambda_i e_i$ $\psi e_i=\mu_i e_i$

$\therefore\varphi(e_1\cdots e_n)=(e_1\cdots e_n)\begin{pmatrix}\lambda_1&&\\&\ddots&\\&&\lambda_n\end{pmatrix}$ $\psi(e_1\cdots e_n)=(e_1\cdots e_n)\begin{pmatrix}\mu_1&&\\&\ddots&\\&&\mu_n\end{pmatrix}$，这两个矩阵可交换，即$\varphi\psi=\psi\varphi$.

$\Leftarrow$ $\varphi\psi=\psi\varphi$，则φ,ψ至少有一个公共的特征向量，设为x_0，$\varphi x_0=\lambda_0 x_0$，$\psi x_0=\mu_0 x_0$，则

记$e_1=\frac{x_0}{\|x_0\|}$，$L_1=\mathrm{span}\{e_1\}$，则由φ,ψ均自伴随算子可知，$L_1^{\perp}$为φ,ψ的不变子空间。

当V的维数$n=1$时，显然成立，假设$n\leq k-1$，($k\geq 2$)时，结论成立，考虑$n=k$的情况。

则由归纳假设可知，

$\varphi|_{L_1^{\perp}}$，$\psi|_{L_1^{\perp}}$分别记为φ_1,ψ_1，则$\varphi_1\psi_1=\psi_1\varphi_1$，$\therefore\exists\varphi_1$和$\psi_1$的公共的由特征向量组成的标准正交基$e_2\cdots e_n$，$\varphi(e_2\cdots e_n)=(e_2\cdots e_n)\begin{pmatrix}\lambda_2&&\\&\ddots&\\&&\lambda_n\end{pmatrix}$ $\psi(e_2\cdots e_n)=(e_2\cdots e_n)\begin{pmatrix}\mu_2&&\\&\ddots&\\&&\mu_n\end{pmatrix}$

$\therefore e_1\cdots e_n$即为满足条件。 □

例6的推论：若$A'=A$，$B'=B$.($\mathbb{R}$上)则A,B可同时正交相似对角化$\Leftrightarrow AB=BA$.

对于酉阵$A^H=A$，$B^H=B$.($\mathbb{C}$上)，则A,B可同时酉相似化$\Leftrightarrow BA=AB$